IT 494 · PHASE 1 · RESEARCH AND PROPOSAL

Persistent Memory for Stateless Models

A research package: proposal, literature, and candidate directions

Justin Hoffman
Illinois State University, MS Computer Science
Supervisor: Dr. Xing Fang

Assembled August 25, 2026. Literature summaries were produced with LLM assistance from the source documents and verified citations; the proposal is the author's own, drafted before the literature review. See the provenance note inside each part.

CONTENTS

Index

PART 1

IT 494 project proposal, pre-research draft

Drafted from my own notes and records for my correction. Not yet reviewed.

What I have running and what it has done

Every conversation I have with an AI is saved as a raw transcript and never edited. At the end of a working session a summary gets written and filed into a tree, organized by area of my life and then by topic. Those summaries roll up, so a higher level can answer a question without reading everything underneath it. A maintenance pass runs on a schedule: it folds in new material, audits a rotating subset of summaries against the original transcripts, and re-summarizes anything whose source changed after it was written. Nothing is deleted. Children fold up into parents and redundant siblings merge, so what has to be read stays bounded while the detail stays recoverable at the bottom. It is wired into my tools, so a new chat starts knowing the map exists and can search it.

This was built inartfully. I had no knowledge of the field when I started, so nothing about it follows a known design. It accreted as I hit problems, and it is a pile of scripts and text files rather than a system anyone would draw on a whiteboard. It is also in daily use and doing real work.

The clearest proof was annual training. I ran separate chats at the same time for the flight schedule, the storyboard, the weather update, and the situation report, all against one shared knowledge base. For the storyboards I pulled images and video out of email and matched them against the mission tracker, AMRS, and the flight schedule by their metadata, then built the boards from a cached template. Each of those threads would have exceeded a single conversation on its own, and they had to agree with each other. The D&D campaign I run is the same shape: long-running, many sessions, dependent on decisions made months ago staying findable and staying correct. When the system gets something wrong, I correct it by hand.

The end state I am aiming at

The model is essentially a function. It takes a prompt and predicts the best answer: a decision engine. But an engine does not make an airplane. The memory system is the airframe, the part that carries state from one call to the next, and the airframe is what I am building. The end state in one sentence: a user level knowledge back end that can be scaled to an organization.

Today the backend serves one user, me. The organizational version I can picture concretely is a chat help desk that learns from its own conversations. The AWS Summit example was a utility answering billing questions with access to account information: store the raw conversations, map how conversations actually progress, track what was concluded, digest it into long-term memory. Many users, one organization. My GAO work is why I think this is tractable rather than speculative. Pulling central ideas out of free human text worked. What failed was asking one tool to discover topics and enforce a rigid classification at the same time; separate those jobs, discovery on one side and the organization’s own

fixed categories on the other, and it works. The utility already has its categories in its dispute codes, the same way the agency had its survey indexes. I also came out of that work with methods for judging whether this kind of output is any good, and that is what I would bring.

The help desk has something my system does not: an outcome signal. Ticket closed, reopened, escalated, repeat contact. My system cannot learn that it gave a wrong answer. It records filing something in the wrong place, but a wrong answer leaves no trace, so there is nothing to measure improvement against. Scale also exposes what my setup leans on. The one writer is me, the one reader is me, and the correction mechanism is my own hand; an organization gets none of those, and the system would have to come loose from my machine and my life's categories. [CHECK: that last sentence extends what you have said; it is not on record in your words]

What I do not know

I have no formal grounding in knowledge graphs or in persistent memory for AI, and I am unlikely to encounter either in a classroom. A model is stateless, so anything that appears to remember is machinery built around it: vector stores, retrieval-augmented generation, caching. I know roughly what these do. I do not know how they compare, where each breaks down, or what else exists. I do not even know that knowledge graphs are the way I want to go.

I do not have real metrics. I set up a fixed set of questions with known answers, and the system answers them, but the set is imperfect and I would not claim it measures much. So I cannot state the system's limitations with any confidence. Problems get patched as they surface, which is fine for my own use and is not evidence of anything.

I do not know whether my structural instincts hold up. I chose written summaries over similarity search on raw text, because a summary carries a judgment about what mattered and what superseded what [CHECK: the choice is yours; unsure this stated reason is in your own words]. I chose a record where facts are never overwritten, because relationships change and deciding what is current is a read-time judgment, not a write-time one. [CHECK: the never-overwrite choice is yours; this stated reason is a reconstruction] I think the real trick is having the AI organize information on its own, a learned database, and I suspect picking the structure is the easy half; the hard half is the system changing its own structure as it learns, underneath material already filed [CHECK: unsure whether that split is your conclusion or one that arrived in conversation]. A drive can be as simple as observe, learn, adapt, improve, but the model does nothing unless directed: my schedulers fire on time, a drive would fire on condition, and my maintenance log already computes conditions it is not allowed to act on. Whether any of this matches how the field thinks about these problems, I cannot say. And one claim under the whole multi-chat use case gets checked before anything else: that concurrent sessions cannot share a context window regardless of its size.

How I would spend the semester

Research first, then the proposal. The goal at this stage is to understand the problem and decide what to investigate, so the near work is gathering resources and reading, and I will not commit to a design before that is done. Where I expect the time to go after that: taking the tree from something that works for me to something validated and tested. That would be a good use of the time, and I currently do not know how to do it. Working out how systems like this are measured, and by what, is what I need the reading for before I can claim anything about mine. One constraint stands regardless of direction: whatever code this project produces, I work through myself for quality control, the same way I correct the tree by hand now. [CHECK: your stated constraint was working through the code on the pieces for QA; the phrasing here is tightened, adjust to taste]

PART 2

Reading list

The working list for Phase 1. Each entry names its reference copy in `papers/` and its summary in `summaries/one-pagers/` by slug. Citations were verified against source pages before inclusion; entries marked *library* need a pull through the ISU library, with the DOI in `papers/MANIFEST.md`.

The advisor-meeting record from 2026-08-19 carries the original 44-item list with fuller annotations. This file supersedes it as the working index and adds the knowledge-graph material requested on 2026-08-25, curated to ten items.

Start here

1. Kumaran, Hassabis, McClelland 2016. What Learning Systems do Intelligent Agents Need? Trends in Cognitive Sciences 20(7). *library* · kumaran2016-cls-updated
2. Park et al. 2023. Generative Agents. UIST 2023. arXiv:2304.03442 · park2023-generative-agents
3. Sarthi et al. 2024. RAPTOR. ICLR 2024. arXiv:2401.18059 · sarthi2024-raptor
4. Rasmussen et al. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 · rasmussen2025-zep
5. Edge et al. 2024. From Local to Global: A Graph RAG Approach. arXiv:2404.16130 · edge2024-graphrag

Consolidation and agent memory

1. McClelland, McNaughton, O'Reilly 1995. Psychological Review 102(3). *library* · mcclelland1995-cls
2. Teyler, Rudy 2007. The hippocampal indexing theory. Hippocampus 17(12). *library* · teyler2007-hippocampal-index
3. Tononi, Cirelli 2014. Sleep and the Price of Plasticity. Neuron 81(1). *library* · tononi2014-shy
4. Zacks et al. 2007. Event perception. Psychological Bulletin 133(2). *library* · zacks2007-event-perception
5. Sumers et al. 2024. Cognitive Architectures for Language Agents. TMLR. arXiv:2309.02427 · sumers2024-coala
6. Rezazadeh et al. 2025. MemTree. ICLR 2025. arXiv:2410.14052 · rezazadeh2025-memtree
7. Talebirad et al. 2026. Toward a Theory of Hierarchical Memory. arXiv:2603.21564 · talebirad2026-hierarchical-theory
8. Wu et al. 2021. Recursively Summarizing Books with Human Feedback. arXiv:2109.10862 · wu2021-recursive-books

Retrieval, vector search, caching

1. Wang et al. 2024. Searching for Best Practices in RAG. EMNLP 2024. arXiv:2407.01219 · wang2024-rag-best-practices
2. Douze et al. 2024. The Faiss library. arXiv:2401.08281 · douze2024-faiss
3. Kuffo, Krippner, Boncz 2025. PDX. SIGMOD 2025. arXiv:2503.04422 · kuffo2025-pdx
4. Malkov, Yashunin. HNSW graphs. TPAMI. arXiv:1603.09320 · malkov2016-hnsw
5. Chan et al. 2025. Don't Do RAG: Cache-Augmented Generation. WWW 2025. arXiv:2412.15605 · chan2025-cag
6. NoLiMa: Long-Context Evaluation Beyond Literal Matching. arXiv:2502.05167 · nolima2025-long-context
7. Jeong et al. 2024. Adaptive-RAG. NAACL 2024. arXiv:2403.14403 · jeong2024-adaptive-rag

Benchmarks and faithfulness

1. Maharana et al. 2024. LoCoMo. ACL 2024. arXiv:2402.17753 · maharana2024-locomo
2. MemoryAgentBench. arXiv:2507.05257 · wu2025-memoryagentbench
3. Chang et al. 2024. BooookScore. ICLR 2024. arXiv:2310.00785 · chang2024-booookscore
4. Kim et al. 2024. FABLES. COLM 2024. arXiv:2404.01261 · kim2024-fables

Knowledge graphs: foundations already on the list

1. Chen 1976. The Entity-Relationship Model. ACM TODS 1(1). *library* · chen1976-er-model
2. Angles et al. 2017. Foundations of Modern Query Languages for Graph Databases. ACM CSUR 50(5). arXiv:1610.06264 · angles2017-graph-query-foundations
3. Hogan et al. 2021. Knowledge Graphs. ACM CSUR 54(4). arXiv:2003.02320 · hogan2021-knowledge-graphs
4. Snodgrass 1999. Developing Time-Oriented Database Applications in SQL. Morgan Kaufmann, author-released PDF · snodgrass1999-time-oriented-db
5. Weikum et al. 2021. Machine Knowledge. FnT Databases. arXiv:2009.11564 · weikum2021-machine-knowledge
6. Pan et al. 2024. Unifying Large Language Models and Knowledge Graphs. IEEE TKDE. arXiv:2306.08302 · pan2024-unifying-llm-kg
7. Rossi et al. 2021. KG Embedding for Link Prediction. ACM TKDD. arXiv:2002.00819 · rossi2021-kg-embedding

Knowledge graphs: added 2026-08-25

Curated to ten from a verified 28-item survey; the remainder are listed at the bottom and available on request. Weighted toward representation and current practice. Knowledge graphs are one candidate representation for this project, not a commitment.

1. Angles 2018. The Property Graph Database Model. AMW 2018, CEUR Vol-2100. What a property graph formally is; the model closest to a tree of pages with typed links · angles2018-property-graph-model
2. Hernandez, Hogan, Krotzsch 2015. Reifying RDF: What Works Well With Wikidata? SSWS at ISWC 2015. The four ways to represent qualified, non-binary facts, compared on real data · hernandez2015-reifying-rdf-wikidata
3. Cai et al. 2024. A Survey on Temporal Knowledge Graph. arXiv:2403.04782. Where facts-valid-for-an-interval lives; representation chapters, not the embedding chapters · cai2024-temporal-kg-survey
4. Rost et al. 2021. Bitemporal Property Graphs to Organize Evolving Systems. arXiv:2111.13499. Valid time and transaction time on every node and edge, so corrections never erase history · rost2021-bitemporal-property-graphs
5. Fowler 2021. Bitemporal History. martinfowler.com. The practitioner bridge to the same idea, readable in twenty minutes. HTML · fowler2021-bitemporal-history
6. Vrandecic, Krotzsch 2014. Wikidata. CACM 57(10). The largest deployed store of time-scoped, source-attributed, supersedable facts · vrandecic2014-wikidata
7. Zhong et al. 2024. A Comprehensive Survey on Automatic Knowledge Graph Construction. ACM CSUR 56(4). arXiv:2302.05019. The construction pipeline end to end · zhong2023-kg-construction-survey
8. Noy et al. 2019. Industry-scale Knowledge Graphs: Lessons and Challenges. CACM 62(8). How Google, Amazon, Microsoft, eBay and LinkedIn actually run them. Free HTML at queue.acm.org/detail.cfm?id=3332266 · noy2019-industry-scale-kgs
9. Xu et al. 2024. RAG with Knowledge Graphs for Customer Service Question Answering. SIGIR 2024 industry track. arXiv:2404.17723. LinkedIn's deployed help-desk graph, directly on use case (a) · xu2024-kg-rag-customer-service
10. Balog, Kenter 2019. Personal Knowledge Graphs: A Research Agenda. ICTIR 2019. The academic niche closest to what I have built · balog2019-personal-kg-agenda

Conversation mining and outcomes

1. Kecht et al. 2021. Event Log Construction from Customer Service Conversations. ICPM 2021 · kecht2021-event-log-nli
2. Gung et al. 2023. Intent Induction from Conversations, DSTC 11. SIGDIAL 2023. arXiv:2304.12982 · gung2023-dstc11-intent

3. De Raedt et al. 2023. IDAS: Intent Discovery with Abstractive Summarization. NLP4ConvAI at ACL 2023 · deraedt2023-idas
4. Wegmann et al. 2022. Same Author or Just Same Topic? RepL4NLP 2022. arXiv:2204.04907 · wegmann2022-style-content
5. Brynjolfsson, Li, Raymond 2025. Generative AI at Work. QJE 140(2). *library* · brynjolfsson2025-genai-at-work
6. Ouyang et al. 2026. ReasoningBank. ICLR 2026. arXiv:2509.25140 · ouyang2026-reasoningbank
7. Liu, Zhang, Choi 2025. User Feedback in Human-LLM Dialogues. EMNLP 2025. arXiv:2507.23158 · liu2025-user-feedback
8. Rezazadeh et al. 2025. Collaborative Memory. arXiv:2505.18279 · rezazadeh2025-collaborative-memory

Shared state across sessions

1. Erman, Hayes-Roth, Lesser, Reddy 1980. The Hearsay-II Speech-Understanding System. ACM CSUR 12(2). *library* · erman1980-hearsay-ii
2. Hayes-Roth 1985. A Blackboard Architecture for Control. Artificial Intelligence 26(3). *library* · hayesroth1985-blackboard-control
3. Salemi et al. 2026. LLM-Based Multi-Agent Blackboard System. arXiv:2510.01285 · salemi2026-blackboard-llm
4. Pollertlam, Kornsuwannawit 2026. Beyond the Context Window. arXiv:2603.04814 · pollertlam2026-beyond-context-window

Cut from the knowledge-graph survey, available on request

Named Graphs (Carroll et al., WWW 2005); RDF-star foundations (Hartig 2017); OWL 2 Profiles tutorial (Krotzsch 2012); the OneGraph vision paper (Lassila et al. 2023); LLM information-extraction survey; LLM-empowered KG construction survey (Bian 2025); entity-resolution overview (Christophides et al.); Open IE survey; Knowledge Vault (Dong et al. 2014); AutoKnow (Amazon, KDD 2020); GQL and SQL/PGQ pattern matching (SIGMOD 2023); a practitioner piece on why KG projects fail; the PKG ecosystem survey and PKG API tool paper; text-lifelog knowledge-base construction; LifeGraph; PIMO.

PART 6

Three shapes the project could take

Prepared by the assistant from the research package, the design brief, and the constraints on record: roughly 70 to 100 project hours this fall concentrated between September 1 and November 15, PhD applications due December 1 to 15, IT 494 continuing through Spring 2027, and the standing requirement that Justin works through all code himself. These are judgments, not options for the sake of options. A recommendation follows the three shapes.

Shape 1. Measure the tree

The semester's product is an evaluation, not a feature. Build the harness that the system has never had: a stratified question set over the archive with four bands (answerable from rollups alone; requiring a specific leaf; requiring synthesis across areas; requiring a fact that was later corrected), scored across delivery arms (summaries preloaded, retrieval as it stands, and both). Add the two measurements the literature does not supply: a counterfactual miss rate, meaning how often the store held the answer and the session never consulted it, and a stale-serve rate, meaning how often a superseded fact was delivered as current. Write the result as a short paper by early December.

What it pays: this is the only shape that produces a December artifact for the applications, and it converts the system's weakest point, no real metrics, into the contribution itself. The fourth band and the miss rate have no published counterpart in the package's corpus. It fits the hour budget because the corpus, the golden-question scaffolding, and the maintenance instrumentation already exist; the work is design, measurement, and writing rather than construction.

What it risks: no new capability ships. The demo at the end is a report and a harness, which is the right artifact for a committee and a thin one for anyone else. The result may also be unflattering to the system, which is a feature academically and still worth saying out loud.

Shape 2. Ship the backend

The semester's product is a distributable system. Extract the engine from the personal archive: the capture, filing, rollup, and maintenance machinery, separated from one man's life categories, installable by someone else, with the multi-chat shared-state pattern from annual training as the flagship demonstration. The organizational use case becomes a design target rather than a hypothetical: one backend, several concurrent consumers.

What it pays: a portfolio artifact that a non-academic audience can run, which serves the federal hiring channel better than a paper, and a foundation the help-desk use case could later stand on.

What it risks: it does not produce a December paper, and the engineering will not fit 70 fall hours under a review-every-line constraint. Packaging, isolation of personal data, and installation paths are exactly the

kind of work that consumes weeks silently. Honestly scoped, this is a spring product, and choosing it for fall means entering the application window with nothing new.

Shape 3. One innovation, studied

Pick the single narrowest mechanism the research pass left unclaimed and study it properly: implement, measure against the obvious alternative, write. The two candidates the package supports are content-derived staleness (staleness recomputed from the material itself rather than asserted by whoever wrote it, which stays correct under hand edits, where flag-based schemes silently fail) and outcome-signal capture (recording, at the moment a session uses the memory, whether the answer survived, which the system currently cannot do). Either is a contained experiment on the live archive.

What it pays: the strongest research identity per hour. A narrow, true, measured claim is a better writing sample than a broad description of a system.

What it risks: single-shot exposure. If the experiment lands null, December holds a null result on a personal corpus, which is publishable in a workshop sense but is a harder sell, and the wider system story goes untold.

Recommendation

Treat the year as two semesters, because IT 494 is two semesters. Fall is Shape 1 carrying Shape 3 inside it: the evaluation harness is the semester, and the corrected-facts band plus the miss rate are the innovation, embedded where they cannot miss the December window. Spring is Shape 2 built on what fall measured: ship the backend whose properties are now documented rather than asserted, with the spring half of the course absorbing the packaging work that would sink the fall.

Two commitments make the recommendation concrete. First, the paper is scoped to what the harness shows by November 15, with the writing weeks aligned to the calendar's dead stretches so coursework collisions are planned rather than suffered. Second, every artifact of the fall, questions, scoring code, and results, is built for reuse as the spring backend's regression suite, so nothing measured is thrown away.

PART 7

End product, test data, and what gets recorded

Prepared by the assistant. This outline assumes the recommendation in the project-shapes document: fall produces a measured evaluation of the memory system with a short paper; spring produces the distributable backend. If a different shape is chosen, the testing section still applies, since every shape eventually answers to the same measurements.

The end product, concretely

Fall delivers three things. First, an evaluation harness in the project repository: a stratified question set over the archive, scoring code, and a runner that evaluates each delivery arm under identical conditions. Second, a results report: the arm-by-arm, band-by-band numbers with their definitions, honest about what the corpus is and how the questions were built. Third, the paper distilled from the report, targeted at a workshop or arXiv by early December.

Spring delivers the backend: the engine separated from the personal archive, installable, documented, with the fall harness repurposed as its regression suite and the annual-training multi-chat pattern as the demonstration.

Test data, the fork that must be chosen deliberately

Three options exist and they answer different questions.

The personal archive is the primary instrument. Two years, roughly a thousand distilled summaries over twenty million tokens of transcript, with a single ground-truth authority available: the person the corpus is about. No published benchmark has this property, and the corrected-facts band is only constructible here, because only this corpus carries documented instances of facts that changed and were superseded. Its weaknesses are equally real: it is private, so no one can reproduce the numbers; and the model that wrote the summaries has seen patterns of the writer's life, so contamination must be named in the report rather than waved off. The mitigation is procedural: questions authored from transcripts by a process that is documented, answers fixed before any arm runs, and the question set published even though the corpus cannot be.

A public benchmark supplies comparability. One suite from the package, run in its published configuration, anchors the harness to numbers other people can check. This is a secondary result and should be scoped as a week, not a month.

A fresh synthetic corpus is the clean option and the expensive one: conversations generated or collected after the fact, never seen by any maintenance pass. It answers the contamination objection completely and costs more hours than the fall has. It belongs in spring, or in the paper's future-work section, stated plainly.

The deliberate choice this outline assumes: primary results on the personal archive with the contamination caveat in the open, one public suite for anchoring, synthetic deferred.

What gets recorded

Every run records the same row: date, arm, configuration hash, question identifier and band, verdict against the fixed answer, tokens in and out, dollars, and latency. Alongside the runs, the standing counters the system already computes stay in the record: coverage, drift and staleness counts, and audit results per maintenance pass, since the paper's claims about currency rest on them. Incidents get their own log: every observed case of the store holding an answer that a session failed to consult, and every case of a superseded fact served as current, each with enough context to be re-derived. These two logs are the raw material of the miss rate and the stale-serve rate, and they start accumulating the day the harness exists, not the day the paper is written.

What gets printed

For the advisor, three artifacts on a cadence. A one-page metric definition sheet, early, so the measurements are agreed before results exist. The arm-by-band results table with confidence noted, once per milestone. And the review trail: which code was walked through line by line, when, and what changed as a result, kept as a running log rather than reconstructed later.

The QA constraint, structurally

Nothing lands unreviewed. Work arrives in units small enough to read in one sitting, each with a stated purpose and a diff against the last reviewed state; the harness runner, the scorer, and every metric definition are review gates, meaning the project does not proceed past them until they have been walked through and signed off in the log. The weekly rhythm follows the campus calendar: build lands Tuesday through Friday, review happens in the Monday and Wednesday campus gaps, which are reading time by construction. The five dead stretches in the design brief are scheduled as review-and-writing weeks, not build weeks, because reading survives a deadline week and building does not.

One-page summaries of the sources

44 summaries, alphabetical by first author. Each states at its foot how much of the source it was written from.

Foundations of Modern Query Languages for Graph Databases

Angles et al. 2017, ACM Computing Surveys 50(5), arXiv:1610.06264

This survey organizes graph querying by feature rather than by language, aiming to bridge database theory and the practical languages SPARQL, Cypher, and Gremlin. It fixes two data models: edge-labelled graphs (the RDF style, edges as label triples) and property graphs, for which the paper gives a new formalization as a tuple (V, E, rho, lambda, sigma) with labels and attribute maps on both nodes and edges. On top of these it builds a small feature hierarchy: basic graph patterns (equivalent to conjunctive queries), complex graph patterns adding projection, join, union, difference, optional, and filter, then regular path queries (RPQs) for arbitrary-length navigation, and their combinations into navigational graph patterns. Each feature is illustrated with worked queries in all three languages.

The load-bearing content is the semantics catalog and the complexity that follows from it. Pattern matching can run under homomorphism semantics (SPARQL, Gremlin), isomorphism variants (Cypher uses no-repeated-edge), or cheaper simulation semantics; path queries can return arbitrary, shortest, simple, or edge-distinct paths, each as bags or sets. Evaluating basic patterns with projection is NP-complete in combined complexity under both homomorphism and isomorphism semantics, but polynomial under simulation, and only logarithmic space in data complexity, where the query is fixed and only the graph grows. Adding optional or difference lifts complex patterns to PSPACE-complete. RPQs returning node pairs under arbitrary-path semantics run in linear time O(|G| times |L|), yet the same query under simple-path semantics is NP-complete even in data complexity. The authors also flag what is unknown: Gremlin is Turing-complete in full, and Cypher's no-repeated-edge semantics and path unwinding (already NP-hard) had, as of 2017, essentially no formal analysis.

For a personal knowledge backend over conversational history, the practical lesson is that the store will be a property graph in all but name (entities, typed relations, provenance attributes on edges), and that the expensive parts are choices, not fate. Fixed, small query templates over a growing graph sit in the logspace data-complexity regime, which is the scaling case an org-wide memory would actually hit; tree-shaped patterns stay tractable where cyclic ones do not (treewidth); and multi-hop recall queries (who told whom, transitively) are the linear-time RPQ case so long as the system returns endpoints rather than enumerating simple paths. The semantics table also explains why memory layers built on Neo4j versus RDF stores can return different answers to the same conceptual query.

Read from: pages 1-25 and 37-41 of 59

The Property Graph Database Model

Renzo Angles, AMW 2018, CEUR Vol-2100 paper 26

This short paper (ten pages) gives a formal definition of the property graph database model, the abstraction behind Neo4j, Amazon Neptune, Azure Cosmos DB, and Titan. Angles observes that despite the commercial dominance of property graphs there is no standard specification of the model, which fragments the market and blocks research that would apply across systems. Following Codd's framing of what a database model must contain, the paper supplies the three components in turn: a data structure (Section 2), integrity constraints (Section 3), and a query language with a Datalog-based semantics (Section 4). It is a definitional paper, not an implementation or evaluation; nothing is benchmarked and no system is built.

The data structure is a tuple G = (N, E, rho, lambda, sigma): finite node and edge sets, an incidence function mapping each edge to an ordered node pair, a partial labeling function assigning sets of labels to nodes and edges, and a partial property function assigning sets of atomic values to (object, property name) pairs. The definition deliberately permits multiple labels per object and multiple values per property. A schema is a second tuple (TN, TE, beta, delta) naming node types, edge types, typed properties, and which edge types are allowed between which node-type pairs; schema-instance consistency is then four validity conditions checked per node, edge, property, and endpoint pair. Mandatory versus optional properties, unique attributes, and cardinality constraints are sketched as extensions rather than fully developed. The query language combines SQL, SPARQL, and Cypher syntax (SELECT, FROM, MATCH, WHERE, plus UNMATCH for safe negation and UNION), and its semantics is given by translation to non-recursive safe Datalog with negation: a graph becomes Node, Edge, Label, and Prop facts (property identifiers exist specifically to support multivalued properties), and each query becomes a set of rules. The translation is shown by worked example only, not proved in general, and the language omits recursion, so path queries, the signature graph operation, are out of scope.

For a personal knowledge backend over conversational history, the schema machinery is the useful part: the delta function, which whitelists edge types between node-type pairs, is exactly the shape of a vocabulary contract for an extraction pipeline writing entities and relations into a store, and the optional-schema stance (all functions partial) matches how such a store grows before its ontology settles. The Datalog encoding also shows that a property graph reduces cleanly to four relational predicates, a useful mental model for judging whether a graph database is buying anything over a relational one at small scale. As a reading of how knowledge graphs are represented, this is the cited formalization of the property graph side, though its query fragment is weaker than what Cypher or the paper's own G-CORE successor provide.

Read from: full text

Personal Knowledge Graphs: A Research Agenda

Balog and Kenter, ICTIR 2019, DOI 10.1145/3341981.3344241

This is a four-page position paper from two Google researchers that defines the personal knowledge graph (PKG) as a distinct research object and lays out an agenda for studying it. The paper contains no system, dataset, or experiment; its method is conceptual. It defines the PKG, contrasts it with prior work (Montoya et al.'s PIM knowledge base, Gyrard et al.'s personalized health graph, Yen et al.'s life-event extraction from tweets, and personalized views over general KGs), then works through four problem themes, each anchored by an explicit research question, and closes with notes on evaluation, implementation, and utilization.

The definition it establishes: a PKG is structured knowledge about entities and relations of personal rather than general importance, with a mandatory "spiderweb" topology in which every node connects, directly or indirectly, to one central user node. Three properties separate PKGs from general KGs: entities need not be globally prominent (a friend, "my guitar", "Mom's dentist"); entity information can be radically sparse, sometimes just a type plus the relation to the user; and relations are often short-lived, the opposite of the stability criterion general KGs use for inclusion. The four research questions cover representation under sparse, ephemeral predicates (RQ1); entity linking when the target entity has little or no structured information and may not be a proper noun, which the authors frame as long-tail linking where linking, population, and NIL-detection intertwine (RQ2a, plus RQ2b on when to link against the PKG versus a general KG); automatic population and maintenance without human curators, where they note that data-hungry neural link prediction is unlikely to transfer (RQ3); and continuous, one-to-many, potentially two-way integration with external sources, with the user in the loop for conflicts (RQ4). On evaluation they are blunt: no open PKG dataset exists, logging real user interactions is costly and privacy-fraught, and synthetic data may be the only path.

For a personal knowledge backend over conversational history, the paper is a checklist of the hard parts rather than a solution: first-mention population, resolving "Jamie" or "my guitar" without any external profile, deciding what to store (only user-relevant attributes, not full entity records), and expiring ephemeral relations all map directly onto conversation-derived memory. The user-centered spiderweb constraint is a concrete schema decision a backend can adopt, though the paper says nothing about multi-user or organizational graphs, where the single central node no longer holds. As a representation reference it is a compact framing of how KGs are used (entity linking, population, verification, completeness prediction) with pointers into that literature, not a treatment of any mechanism in depth.

Read from: full text

A Survey on Temporal Knowledge Graph: Representation Learning and Applications

Cai, Mao, Zhou, Long, Wu, and Lan; arXiv preprint 2403.04782, 2024.

This is a survey of temporal knowledge graph representation learning (TKGRL): methods that learn low-dimensional embeddings for entities, relations, and timestamps in graphs whose facts are quadruples (head, relation, tail, timestamp) rather than static triples. The authors position it as the first broad survey of the area; prior surveys covered static KGs with a short temporal section, or only the completion subtask. The paper proceeds from problem formulation, datasets, and metrics, to a ten-category taxonomy of methods, to three application areas, and closes with future directions. The taxonomy chapters catalog roughly forty models from 2017 to 2024 at the level of representation space, encoder, and scoring function, without new experiments or head-to-head benchmark numbers, so the survey establishes structure rather than empirical rankings.

The standing infrastructure it documents is concrete. Four benchmark families dominate: ICEWS event data (ICEWS14 has 7,128 entities, 230 relations, 365 timestamps, 90,730 facts), GDELT (2,278,405 facts over 2,751 timestamps), Wikidata, and YAGO; evaluation is by mean reciprocal rank and Hit@k. The ten method categories are transformation (translation and rotation, extending TransE and RotatE with time hyperplanes or temporal rotations), tensor decomposition (order-4 CP and Tucker variants such as TComplEx and TuckERT), graph neural networks, capsule networks, autoregression (RE-NET, RE-GCN: snapshot sequences encoded by relational GCNs plus GRUs), temporal point processes for irregular intervals (Know-Evolve, Graph Hawkes), interpretability via subgraph reasoning or reinforcement learning, language-model methods (in-context forecasting, retrieval plus fine-tuning), few-shot learning for rare entities and relations, and a residual class including copy-generation and neural ODEs. A useful task split runs throughout: interpolation (completing missing past facts) versus extrapolation (forecasting future ones), which demand different machinery.

For a personal knowledge backend built over conversational history, the relevant contribution is the framing itself: facts that hold only during an interval, the quadruple representation, and the interpolation versus extrapolation distinction map directly onto memory that must record when something was true and predict what still is. The autoregressive snapshot view suggests treating accumulated conversation state as a timestamped graph sequence, and the few-shot chapter addresses exactly the sparse new entities a personal graph accumulates. The scalability discussion is candid that current benchmarks are far smaller than real-world graphs and that most methods ignore scaling, so organizational use remains an open problem; the LLM-integration section (using language models for relation descriptions and semantic enrichment) is directional rather than settled. As a map of how temporal knowledge is represented and scored, the survey is a serviceable entry point and reference index rather than a source of design-ready results.

Read from: full text, pages 1 through 25 of 36 (everything before the references), with the per-model embedding sections in chapter 3 skimmed rather than studied.

Don’t Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks

Chan, Chen, Cheng, and Huang; WWW Companion 2025; arXiv:2412.15605

The paper proposes cache-augmented generation (CAG) as a replacement for retrieval-augmented generation when the knowledge base is small enough to fit in a long-context model’s window. Instead of retrieving passages per query, the whole document collection is preprocessed once into the model’s key-value cache, which is stored on disk or in memory; at inference time only the query is appended, and the model answers with the entire corpus already resident in its inference state. The method has three phases: offline preloading (a one-time encoding cost regardless of query count), inference over the cached context, and a cache reset that truncates the appended query tokens rather than reloading from disk. The claimed gains are zero retrieval latency, no retrieval errors, and a simpler architecture with no retriever to tune.

Experiments use Llama 3.1 8B on subsets of SQuAD and HotPotQA sized from 21k to 85k tokens (3 to 64 documents), against BM25 sparse retrieval and OpenAI dense embeddings via LlamaIndex at top-1 through top-10, scored with BERTScore. CAG wins in most configurations: on HotPotQA-small it scores 0.7951 against 0.7676 for the best sparse RAG setting, and on SQuAD-large 0.7734 against 0.7658. The margin narrows as the corpus grows; at HotPotQA-large CAG’s 0.7407 falls below sparse RAG top-5 at 0.7535, which the authors connect to known long-context degradation. Timing on HotPotQA shows CAG generation at 0.85 to 2.26 seconds across sizes, while plain in-context learning (encoding the reference text per query) takes 9.3 to 92.1 seconds, a roughly 40x gap at the large size. Sparse RAG beating dense RAG throughout suggests the benchmarks were not retrieval-hard. All results are for one 8B model on corpora under 100k tokens; the authors state plainly that the method is impractical for significantly larger datasets, naming small-company knowledge bases, FAQs, and call centers as the intended fit.

For a personal knowledge backend over conversational history, CAG marks one endpoint of a design spectrum: below some corpus size, retrieval infrastructure is pure overhead and precomputing the context is both faster and more accurate. A personal archive might start under that threshold; an organizational one will not, and the paper’s own large-corpus result shows the accuracy cost of stuffing everything into context. Its concluding suggestion, a preloaded foundation context with retrieval reserved for edge cases, is the more durable idea for memory systems: a tiered design where hot, stable knowledge lives in cache and the long tail stays behind a retriever. The precomputed-KV technique itself is memory-adjacent engineering worth tracking, since it treats an LLM’s inference state as a persistable artifact.

Read from: full text

BooookScore: A Systematic Exploration of Book-Length Summarization in the Era of LLMs

Chang, Lo, Goyal, and Iyyer, ICLR 2024, arXiv:2310.00785

The paper studies how well LLMs summarize books longer than 100K tokens, a task that forces the document to be chunked and the chunk summaries combined. The authors compare two prompting workflows: hierarchical merging, where chunk summaries are recursively merged until one remains, and incremental updating, where a running global summary is updated and compressed as the model steps through the book. Because existing benchmarks like BookSum sit in LLM pretraining data, they build a contamination-resistant corpus of 100 recently published books (average 190K tokens, no public summaries found for any of them) and collect 1,193 span-level human annotations on GPT-4 summaries, at a cost of about $3K and 100 annotator hours. From these they derive a taxonomy of eight coherence error types, including two not seen in earlier taxonomies for fine-tuned summarizers: causal omissions and salience errors. They then automate the annotation as BooookScore, a GPT-4 prompt that checks each summary sentence against the taxonomy; the metric is the fraction of error-free sentences.

Validation shows the automatic annotations have 78.2 percent precision against human judgment versus 79.7 percent for the human annotations themselves, and system-level scores computed from human and LLM annotations nearly coincide. Applying the metric across models (a $10K API study), Claude 2 and GPT-4 lead (hierarchical scores of 91.1 and 89.1 at chunk size 2048), Mixtral-8x7B roughly matches GPT-3.5-Turbo, and LLaMA2-7B-Instruct trails badly at 72.4 with 36 percent repeated trigrams and cannot perform incremental updating at all. Hierarchical merging is almost always more coherent than incremental updating, but the exception is instructive: Claude 2 with an 88K chunk jumps from 78.6 to 90.9 on incremental updating, meaning fewer update-and-compress steps largely erase the gap. Annotators still preferred incremental summaries for detail (83 to 11 percent) while preferring hierarchical ones overall (54 to 44), and overall preference did not correlate with the coherence score. The taxonomy and metric are both derived solely from GPT-4 outputs on English trade books, a scope limit the authors flag themselves.

For a personal knowledge backend built over conversational history, this is essentially a study of the two compression strategies such a system must choose between. Incremental updating is exactly the rolling-summary pattern most memory systems use, and the paper shows it degrades coherence through repeated compression unless each update window is large; hierarchical merging is more coherent but loses long-range dependencies and detail. The finding that omission errors (entity, event, causal) dominate the error distribution tells a memory designer what to audit for, and the reference-free, taxonomy-grounded LLM-judge protocol is a workable template for evaluating memory summaries where no gold answer exists.

Read from: pages 1-12 of 35 (full main text; pages 13-35 are references and appendices)

IDAS: Intent Discovery with Abstractive Summarization

De Raedt et al. 2023, IDAS: Intent Discovery with Abstractive Summarization, NLP4ConvAI at ACL 2023

The paper tackles intent discovery: partitioning a pile of unlabeled utterances into clusters that each share a conversational goal, the usual first step in building a chatbot without paying for annotation. Prior methods train an unsupervised sentence encoder on the domain data, then cluster; the authors argue that what makes two encodings similar is opaque and often driven by syntax or noun overlap rather than intent. IDAS instead keeps the encoder frozen and does the disambiguation in text: an LLM (GPT-3 text-davinci-003, temperature 0) abstractively summarizes each utterance into a short label such as “when is next payday”. Because the task has no labeled examples for in-context learning, IDAS bootstraps them. It runs an initial K-means, takes the utterance nearest each centroid as a prototype, has the LLM label the prototypes with a five-word-max instruction, then labels every remaining utterance using its 8 nearest already-labeled neighbors as ICL demonstrations, growing the demonstration pool as it goes. Utterance and label encodings are averaged, smoothed over n’ nearest neighbors (n’ picked by silhouette score), and clustered with K-means at the true K.

Against MTP-CLNN, the unsupervised state of the art, IDAS gains an average of 3.94 ARI, 2.86 NMI, and 3.34 accuracy points across Banking (77 intents), StackOverflow (20 topics), and a private 1,257-utterance transport dataset (42 intents), with the largest single gain +7.42 ARI on Transport; on Banking the two methods are roughly tied when compared in matched settings. On CLINC (150 intents) IDAS reaches 79.02 ARI and 85.48 accuracy with zero labels, beating two semi-supervised methods that were given labels for 50% of the intents, and trailing only at the 75% ratio. Ablations show every strategy using generated labels beats raw utterance encodings (by 5 to 19 ARI points), nearest-neighbor demonstration selection clearly beats random, and doubling K in the prototype step barely hurts. The results all assume the true intent count is known, and the test sets are small (1,000 to 3,080 utterances); smaller models (curie, babbage, Flan-T5-XL) produced labels too poor to use.

For a personal knowledge backend over conversational history, the transferable idea is that summarize-then-embed beats embed-raw-text when you need clusters organized by meaning rather than surface form, and that a seed-and-grow labeling loop turns an unlabeled corpus into its own demonstration set. The K-must-be-known and per-utterance-LLM-call costs are exactly the constraints that bite at organizational scale, and the paper flags both as open problems.

Read from: full text

The Faiss Library

Douze et al., arXiv preprint, 2024, arXiv:2401.08281

This paper is the design document for Faiss, Meta's open-source C++ library (with Python bindings) for approximate nearest neighbor search over embedding vectors. Rather than proposing a single new algorithm, it explains how the library organizes a dozen index types around two tools, vector compression and non-exhaustive search, and how a user should trade off speed, memory, and accuracy under a fixed budget. The authors frame the whole problem as an "embedding contract": the embedding model makes distances meaningful, and the index's only job is to approximate exact search under the agreed metric. They are equally explicit about what Faiss is not: it extracts no features, runs as no service, and provides no database machinery such as sharding, transactions, or concurrent writes.

The benchmarks establish practical selection rules. On datasets from SIFT features (BIGANN, 128 dimensions) up to 768-dimensional Contriever text embeddings of English Wikipedia, additive quantizers win at small code sizes (around 8 bytes per vector), product-additive hybrids take over at larger budgets, and plain scalar quantizers suffice for long codes. For non-exhaustive search, graph indexes (HNSW, NSG) are recommended below roughly 1M vectors when memory is unconstrained, while inverted-file (IVF) indexes are the only option once compression is needed to fit in RAM; graph quality peaks at 64 edges per node and degrades beyond. Fitting a scaling model on BigANN at 1B vectors, search time grows as N to the 0.29 at 50 percent recall@1 and N to the 0.45 at 99 percent, so cost stays below the square root of N but climbs steeply with the accuracy target. On Deep10M under ANN-benchmarks, Faiss beats SCANN and roughly matches DiskANN's Vamana. A production case study indexes 1.5 trillion 144-dimensional vectors at 54 bytes each (PCA to 72 dimensions plus a 6-bit scalar quantizer), sharded across hundreds of servers into an 83 TiB memory map, reaching about 1 second per query.

For a personal memory backend over conversational history, the field guide here is the sizing logic: a personal corpus sits far below the 10k-vector threshold where indexing beats brute force, while an organizational one crosses into graph-index territory, so the same Faiss code covers both ends without redesign. The gaps matter too. Faiss stores no metadata, filters only through id-bit tricks, and its graph indexes handle deletion poorly; a memory system needs a database layer on top, which is exactly the Faiss-plus-DBMS split that Milvus and Pinecone embody. The retrieval-augmented language model work it cites (Atlas, kNN-LM, RA-DIT) is the direct lineage of persistent AI memory.

Read from: full text

From Local to Global: A GraphRAG Approach to Query-Focused Summarization

Edge et al. 2024, From Local to Global: A Graph RAG Approach to Query-Focused Summarization, arXiv:2404.16130

The paper targets a failure mode of conventional vector RAG: questions that require global understanding of a whole corpus ("what are the main themes here?") are query-focused summarization tasks, not retrieval tasks, and retrieving a handful of similar chunks cannot answer them. GraphRAG's answer is to spend indexing-time compute building structure. An LLM extracts entities, relationships, and factual claims from 600-token chunks, aggregates the duplicates into a weighted knowledge graph, then partitions that graph with hierarchical Leiden community detection. Each community, at every level of the hierarchy, gets a pre-written LLM summary, with higher levels built from the summaries below them. At query time the community summaries are shuffled, chunked, mapped in parallel to partial answers with self-rated helpfulness scores, and reduced into one global answer.

The evaluation compared four community levels (C0 root through C3 leaf) against direct map-reduce summarization of source text (TS) and vector RAG (SS), on two roughly one-million-token corpora: podcast transcripts (1,669 chunks, graph of 8,564 nodes and 20,691 edges) and news articles (3,197 chunks, 15,754 nodes). With 125 persona-generated sensemaking questions per dataset and GPT-4 as judge, every global condition beat vector RAG on comprehensiveness (72 to 83 percent win rates, $p<.001$) and diversity (62 to 82 percent), while vector RAG won the directness control criterion, as expected of shorter answers. A second experiment counting and clustering extracted factual claims (47,075 claims via Claimify) broadly confirmed the LLM-judge results, agreeing on 78 percent of non-tied comprehensiveness comparisons. The efficiency finding matters most: root-level C0 answers used 97 percent fewer context tokens than TS (26,657 versus about a million for podcasts) while still beating vector RAG at 72 and 62 percent. The authors state plainly that all of this is established only for these two corpora, this token scale, and GPT-4.

For a personal knowledge backend over conversational history, the transferable idea is the shape of the index: entities and relationships distilled once, then summarized at nested levels of abstraction, so that broad questions read a few cheap high-level summaries instead of re-digesting raw text. That is essentially a mechanized version of a summarize-once, query-summaries-forever memory tree, and the C0 token numbers quantify what the hierarchy buys. The paper also contributes evaluation machinery (persona-driven question generation, comprehensiveness and diversity criteria, claim-based validation) that applies directly to measuring any persistent memory system where no gold answers exist. Indexing cost is the caveat: 281 minutes of gpt-4-turbo calls for one million tokens, so incremental update strategy is left open.

Read from: pages 1-16 of 26 (full main text and references; appendices skipped)

Bitemporal History

Martin Fowler, martinfowler.com, April 2021, https://martinfowler.com/articles/bitemporal-history.html

This is a pattern essay, not an empirical study. Fowler works a single payroll example through the whole piece: on February 25 the company pays Sally her recorded $6000 monthly salary; on March 15 HR reports that she was actually raised to $6500 back on February 15; on April 5 HR corrects the raise to $6400. He uses the example to motivate treating time as two dimensions. Actual history is what happened in the world given perfect information; record history is what the system believed at each moment. He prefers the terms actual and record over the Snodgrass valid and transaction terminology used in SQL:2011, on the grounds that workshop audiences found the latter confusing. The essay proceeds through tables and plots (actual time on one axis, record time on the other), then generalizes and closes with implementation options.

The concrete machinery is small but exact. A bitemporal query takes two time parameters, in his notation sally.salaryAt('2021-02-25', '2021-03-25'), read as what we believed on March 25 her February 25 salary was. Horizontal slices of the two-axis plot give the actual history as known at some record time; vertical slices give how belief about one date evolved. The key structural claim is that while actual history stops being append-only once retroactive changes are allowed, record history stays append-only: corrections are layered on rather than overwritten, which yields a reliable audit trail of the modifications themselves. He is explicit about scope limits: bitemporality complicates a system significantly, is avoidable when retroactive change is forbidden or when actions capture their inputs (the check records the salary it used), and by itself cannot compute the downstream consequences of a correction, such as back pay or tax effects. For storage he sketches two schemes, a relational table with paired record and actual date ranges, and event sourcing where each event carries both an actual and a record timestamp and simpler read models are projected from the log.

For a knowledge backend built over conversational history, the bearing is direct. Facts extracted from conversations get corrected later, and the record dimension is what lets the system answer both what is true and what was believed when an action was taken, without destroying the audit trail. Fowler's generalization of record time into perspectives, where HR and Payroll hold different record histories of the same fact and a perspective can even be a counterfactual scenario, maps onto multi-agent or multi-user ingestion at organizational scale, though he cautions that stacking many perspective dimensions is rarely worth the complexity. This is design guidance from one practitioner's example, with no benchmarks or formal treatment, so it constrains schema choices rather than settling them.

Read from: full text

Intent Induction from Conversations for Task-Oriented Dialogue Track at DSTC 11

Gung et al. 2023, Intent Induction from Conversations (DSTC 11), SIGDIAL 2023, arXiv:2304.12982

This paper, from AWS AI Labs, describes a shared-task benchmark rather than a single method: the DSTC 11 challenge track on inducing customer intents from raw conversation transcripts. The authors argue that prior intent-mining work evaluated on repurposed classification datasets (BANK77, CLINC150, SNIPS) that lack full dialogues and realistic intent skew. They release three simulated call-center corpora: Insurance (948 conversations, 22 intents, used for development) and Banking (1,000 conversations, 29 intents) and Finance (2,000 conversations, 39 intents) for evaluation, averaging 59 to 71 turns per conversation, three to five times longer than MultiWOZ or SGD. Two subtasks are defined. Task 1 is intent clustering: assign a cluster label to each pre-tagged intentful turn, with the number of reference intents withheld (only a 5 to 50 range given). Task 2 is open intent induction: no turn labels, only noisy automatic InformIntent predictions; systems must produce a labeled training set for an intent classifier, and evaluation trains a logistic regression classifier on frozen all-mpnet-base-v2 embeddings and scores its predictions on an independent test set. Clustering accuracy via Hungarian matching is the primary metric.

Thirty-four teams entered Task 1 and 19 entered Task 2, all using clustering-based pipelines. The winner, T23, reached 69.8 ACC on Task 1 and 76.3 on Task 2, against baselines (k-means over sentence embeddings, with silhouette-based selection of k) at 55.8 and 63.6. Deep embedded clustering variants, particularly SCCL, dominated the top ten; nine of ten top systems used them, and DSE-RoBERTa-large plus supervised pre-training on public intent data marked the winner. Two analyses qualify the numbers. HDBSCAN systems that left noise points unassigned were punished by clustering metrics: propagating labels to T11's noise instances moved it from 33rd to 9th, exposing a weakness of clustering-style evaluation itself. And rerunning Task 2 scoring under nine different classifier encoders left the top ten ranks stable but shuffled ranks 11 through 15, so prediction-based evaluation carries measurable noise. All results are scoped to simulated (not genuinely live) English conversations in three financial-adjacent domains.

For a personal knowledge backend over conversational history, the relevant lesson is methodological. Task 2's framing, judge an induced schema by the downstream system it trains rather than by cluster purity on the source turns, is directly transferable to evaluating induced topic or era structures over an archive: a smaller, cleaner distillation can beat exhaustive coverage. The corpus statistics also matter at organizational scale: real request distributions are long-tailed, so any memory taxonomy induced from usage will concentrate on a few head topics, and the granularity trade-off the metrics here are built around (too many fine categories versus too few coarse ones) is the same one a schema over conversation history must manage.

Read from: pages 1-9 of 18 (full main text and results; remainder is references and appendices)

Reifying RDF: What Works Well With Wikidata?

Hernandez, Hogan, Krotzsch; SSWS workshop at ISWC 2015

The paper asks how best to represent Wikidata, whose statements carry qualifiers and references, in plain RDF so that off-the-shelf SPARQL engines can query it. Since a Wikidata statement annotates an entire subject-predicate-object relation, the authors first reduce the problem to encoding quads (s, p, o, i), where i is a statement identifier that functionally determines the triple; qualifiers then attach to i as ordinary triples. They compare four established encodings: standard reification, which spends three triples per quad on r:subject, r:predicate, and r:object; n-ary relations, the Erxleben et al. scheme Wikidata itself was using, which needs 2(n + m) triples for n quads over m distinct properties; singleton properties, which mint a fresh predicate per statement and need only 2n triples; and named graphs, which store the quad directly. They build all four datasets from a February 2015 Wikidata dump (57.1 million quads plus 237.6 million triples common to every format, so between 57 and 171 million model-specific tuples) and run 14 benchmark queries, expanded into model-specific versions, against 4store, BlazeGraph, GraphDB, Jena TDB, and Virtuoso.

The empirical picture is uneven in an instructive way. Singleton properties, the most concise encoding, broke four of the five engines: 4store and GraphDB could not index the millions of unique predicates, and Jena timed out on every query. 4store also failed to load the named graphs dataset. Only n-ary relations and named graphs could express the SPARQL property paths two queries required. Virtuoso handled all four models reliably; across all engines, named graphs was fastest in 17 of 70 engine-query cases, with standard reification and n-ary relations at 16 each, so no clear winner emerged among those three. The authors note the caveats themselves: cold-cache runs only, a 60-second timeout, one hardware setup, and a preliminary scope. They also observe that 108 two-triple encodings are possible in principle; only the well-known four were tested.

For a personal knowledge backend over conversational history, the load-bearing problem here is the same one: facts need provenance, temporal scope, and confidence attached to them, and the choice of how to reify those annotations determines which stores and query features remain usable. The concrete lessons transfer directly. Elegant-on-paper schemes can violate engine assumptions (predicate cardinality being the killer for singleton properties), quads with a statement identifier are a clean conceptual core, and the identifier join is highly selective, so the extra joins that reification adds cost less than expected but do complicate query planning. The paper is also a compact tour of how a real large knowledge graph, Wikidata at 65 million statements in 2015, actually structures qualified claims.

Read from: full text

Knowledge Graphs

Hogan et al., ACM Computing Surveys 54(4), 2021, arXiv:2003.02320

This is a tutorial survey by seventeen authors, grown out of a Dagstuhl seminar, that gives the field of knowledge graphs its first unified introduction. The authors adopt a deliberately inclusive definition: a knowledge graph is a graph of data meant to accumulate and convey knowledge of the real world, with nodes as entities and edges as relations. From there the paper walks through the full stack: graph data models (directed edge-labelled graphs such as RDF, property graphs as in Neo4j, heterogeneous graphs, and named-graph datasets), query languages (SPARQL, Cypher, Gremlin, G-CORE) and their shared primitives from basic graph patterns up through regular path queries, then schema, identity, and context, deductive reasoning with ontologies and rules, inductive methods, and the lifecycle of creation, quality assessment, refinement, and publication. A running example, a Chilean tourism knowledge graph, carries the formalism.

The paper's substance is organizational rather than experimental, but it fixes useful distinctions and facts. It separates three kinds of schema: semantic (RDFS and OWL, which license inferences under the open-world assumption), validating (ShEx and SHACL shapes, which check completeness of specified portions of the graph), and emergent (quotient graphs computed from latent structure). On identity it treats IRIs, owl:sameAs links, datatypes, and blank nodes as the machinery for saying when two nodes are the same thing. Its practice section quantifies the landscape as of 2021: Freebase held over three billion edges when it went read-only in March 2015, largely migrated to Wikidata; Cyc represents over 900 person-years of manual effort yielding 2.2 million facts and rules; NELL extracted 120 million edges from web text. Drawing on Noy et al., it lists six traits enterprise graphs (Google, Amazon, Facebook, LinkedIn, Bloomberg) share, including billion-edge scale, continuous refinement, and deliberately lightweight taxonomic ontologies rather than heavy formal ones.

For a personal knowledge backend over conversational history, the load-bearing ideas are the ones the paper opens with: graphs let you postpone schema definition while data and scope evolve, which is exactly the position of a system ingesting unpredictable conversations; the identity machinery answers when two mentions are one entity; and the context section (temporal validity, provenance, geographic scope) is the vocabulary for statements true only at a time or per a source. The framing of a knowledge graph as an ever-evolving shared substrate of knowledge within an organisation or community, plus the enterprise-scale trait list, is a direct sketch of what a personal store scaled to an organization would have to become, and the note that industry favors lightweight taxonomies over expressive ontologies is a useful caution against over-engineering the schema.

Read from: pages 1-20 and 74-78 of 135

Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity

Jeong et al., NAACL 2024, arXiv:2403.14403

Adaptive-RAG routes each incoming question to one of three answering strategies based on a predicted complexity level: no retrieval at all (the LLM answers from parametric knowledge), a single retrieve-then-read step, or an iterative multi-step loop that interleaves chain-of-thought reasoning with repeated retrieval. The router is a small T5-Large classifier (770M parameters) that maps a query to label A, B, or C before any answering begins, so the LLM and retriever themselves never change. Since no query-complexity dataset exists, the authors build training labels automatically two ways: run all three strategies on sampled queries and assign the label of the cheapest strategy that got the answer right, then fill remaining gaps using dataset provenance (single-hop benchmarks get B, multi-hop benchmarks get C). They train on about 400 queries per dataset and evaluate on a mixed pool of three single-hop sets (SQuAD, Natural Questions, TriviaQA) and three multi-hop sets (MuSiQue, HotpotQA, 2WikiMultiHopQA), 500 test queries each, with BM25 as the retriever throughout.

The headline result, on GPT-3.5-Turbo-Instruct averaged over all six datasets, is 50.91 F1 at 1.03 retrieval-and-generate steps per query, essentially matching the always-iterate multi-step baseline (50.87 F1) at roughly a third of its 2.81 steps, and beating binary adaptive baselines like Mallen et al.'s Adaptive Retrieval (48.20) and Self-RAG (22.98, though trained on a different base model). The pattern holds for FLAN-T5-XL (3B) and XXL (11B). The efficiency stakes are concrete: 0.35 seconds per no-retrieval query versus 27.18 for multi-step. Two caveats travel with these numbers. The classifier is only 54.52 percent accurate over the three labels, and an oracle router reaches 62.80 F1 at half a step per query, so routing quality, not the strategies, is the current ceiling. Classifier size barely matters; a 60M model performs within about one F1 point of the 770M one.

For a personal knowledge backend over conversational history, the transferable idea is that retrieval depth should be a per-query decision made before retrieval starts, by a component far cheaper than the answering model. Most lookups against a memory tree are single-hop ("what did we decide about X"), some need iterative traversal across sessions, and many need no lookup at all; paying multi-hop cost uniformly is exactly the waste this paper quantifies. The self-labeling trick also matters: usage logs can label their own routing data by recording which strategy sufficed, no annotation budget required. The scope limit is real, though: results cover factoid QA over Wikipedia-style corpora with BM25, not dense retrieval over informal conversational text.

Read from: pages 1-11 of 15

Event Log Construction from Customer Service Conversations Using Natural Language Inference

Kecht, Egger, Kratsch, and Roglinger; ICPM 2021; doi:10.1109/ICPM53251.2021

The paper builds standardized process-mining event logs out of raw customer service conversations, using natural language inference instead of a trained classifier. Each customer inquiry or agent reply becomes the premise of an NLI pair; the hypothesis is a short sentence naming a candidate topic or activity, such as “This example asks for iOS version.” Facebook’s pre-trained BART model, called through the transformers library, returns an entailment probability for every message-hypothesis pair. The workflow has five steps: define domain topics and activities, hand-label a small sample (100 to 300 messages) with binary tags, run five-fold stratified cross-validation to pick a per-label decision threshold from candidates between 0.70 and 0.999 by maximizing the Matthews correlation coefficient, optionally rewrite hypotheses that score poorly, then classify the full corpus and export it as an IEEE XES log via PM4Py. No model training happens anywhere; this is zero-shot classification in the sense of Yin et al. 2019.

Evaluation uses English Twitter support conversations from AmazonHelp (288,828 tweets), AppleSupport (231,683), and SpotifyCares (88,774) after filtering, with a keyword matcher as baseline. With tuned hypotheses, 55 of 56 topic and activity classifications reached an MCC above 0.72 on the labeled samples; on 100 AppleSupport outbound tweets, four of seven activities scored a perfect 1.00 across all metrics. Hypothesis wording mattered a great deal: “This example is about prime” hit 0.91 MCC where the default phrasing managed 0.53, and one grammatically broken default hypothesis scored 0 where the fixed version scored 1. Keywords sometimes sufficed on their own (URL detection was perfect because every Twitter link starts with the same prefix). Three discovery algorithms (Alpha, Inductive, Heuristics Miner) recovered the same coherent process model from the resulting log, so the logs are usable, though the authors concede the mined model covers only the topics they thought to define and analysis stopped at each conversation’s first message.

For a personal knowledge backend over conversational history, the transferable finding is the labeling economics: roughly 300 hand-labeled examples plus threshold tuning were enough to structure 200,000-plus messages, because the pre-trained model carries the semantics and only the decision boundary needs calibration. That pattern (define a schema of event types, verify on a tiny sample, sweep the archive) maps directly onto turning a conversation archive into a queryable timeline of typed events, and adding a new event type never degrades existing ones since each label is inferred independently. The caveats transfer too: results come from short English tweets in three consumer-support domains, hypothesis phrasing is brittle and opaque, and the schema must be known in advance, which is exactly the part a memory system over open-ended conversations cannot assume.

Read from: full text

FABLES: Evaluating faithfulness and content selection in book-length summarization

Kim et al., COLM 2024, arXiv:2404.01261

The paper reports the first large-scale human evaluation of faithfulness in summaries of book-length documents (mean 121K tokens). The authors sidestep two chronic problems at once: data contamination, by using 26 fictional books published in 2023 or 2024, and annotation cost, by hiring 14 Upwork annotators who had already read the books for pleasure. Five models (GPT-3.5-Turbo, GPT-4, GPT-4-Turbo, Mixtral, Claude-3-Opus) each summarize every book via hierarchical merging; GPT-4 decomposes each summary into atomic, decontextualized claims; annotators label each claim faithful, unfaithful, partially supported, or unverifiable, with quoted evidence. The result is 3,158 annotated claims across 130 summaries, collected for $5.2K, about $40 per summary.

Three findings stand out. First, Claude-3-Opus was the most faithful summarizer at 90.9% faithful claims, ahead of GPT-4 and GPT-4-Turbo (about 78%), GPT-3.5-Turbo (72%), and Mixtral (69%); on this corpus of 26 recent novels, only five of 130 summaries were entirely accurate. Second, unfaithful claims cluster around character or relationship states (38.6%) and events (31.5%), and half require indirect, multi-hop reasoning over the narrative to refute, which distinguishes the task from entity-centric fact checking. Third, automatic verification largely fails: on a seven-book subset (723 claims), the best auto-rater, Claude-3-Opus given the entire book as context, reached only 58.2 F1 on detecting unfaithful claims, and it beat both BM25 retrieval (24.9 F1) and human-annotated evidence spans. Beyond faithfulness, coding of the summary-level comments yielded a taxonomy of omission errors: 33% to 65% of summaries omit key events, every model makes chronology errors, and the long-context models systematically over-weight book endings (at least 20% of Claude-3-Opus summaries).

For a personal knowledge backend built over conversational history, the negative results matter most. Retrieval-then-verify, the backbone of most memory and RAG validation schemes, collapses when the query is a claim about states, relationships, or arcs distributed across a long record rather than a localizable fact; even full-context reading by the strongest model of early 2024 could not reliably catch fabrications. A memory system that summarizes long histories into durable records will inherit exactly these failure modes: silent omission of key events, recency bias toward the end of the record, and confident state claims that no single passage can confirm or deny. The paper argues, credibly, that claim-level verification over long documents is itself a benchmark for long-context understanding, and its human-annotation protocol (readers who already know the source) is a workable template for auditing any summarization layer in a memory pipeline. Scope caveats: fiction only, English, 26 books, February 2024 model checkpoints.

Read from: pages 1-14 of 41 (full main text and conclusion; appendices skipped)

PDX: A Data Layout for Vector Similarity Search

Kuffo, Krippner, Boncz 2025, SIGMOD 2025, arXiv:2503.04422

The paper proposes PDX (Partition Dimensions Across), a storage layout for embedding collections that groups vectors into blocks and, within each block, stores each dimension's values contiguously, the way Parquet rowgroups hold columns. On top of it sits PDXearch, a search framework that computes distances dimension-by-dimension across 64 vectors at a time in tight scalar loops that compilers auto-vectorize. The design targets a specific weakness of the standard vector-by-vector layout: pruning algorithms such as ADSampling and BSA, which stop reading a vector's dimensions once a partial distance shows it cannot reach the top-k, interleave bounds checks with distance math and lose their advantage against plain SIMD linear scans. PDXearch scans an IVF bucket in three phases (a full scan of the first block to set a threshold, a warmup that fetches dimensions in exponentially growing steps, and a prune phase that skips discarded vectors once fewer than about 20 percent remain), keeping the code branchless.

On four CPUs (Intel Sapphire Rapids, AMD Zen4 and Zen3, Graviton4) and ten datasets from 16 to 1536 dimensions, the auto-vectorized PDX distance kernels averaged 2.0x over hand-written SIMD kernels on the horizontal layout, and 5.5x when 8 or fewer dimensions are read, the case pruning creates. In IVF index search on Zen4 at the highest recall, ADSampling on PDX ran 4.6x faster than its own SIMD horizontal version and 2.2x faster than FAISS IVF_FLAT, reaching 7.2x over FAISS on the OpenAI/1536 dataset on Intel. The paper also shows that horizontally stored ADSampling can lose to FAISS outright, so the pruning idea only pays off given the right layout. A companion algorithm, PDX-BOND, orders dimensions by how far the query sits from each dimension's block mean; it needs no preprocessing or transformation of the vectors and is exact, and it beat FAISS, USearch, and Milvus on exact search by 2.5x, 3.7x, and 4.0x on average. All results are single-threaded, in-memory, float32, on collections of 0.3 to 10 million vectors; graph indexes like HNSW are discussed but untested.

For a personal knowledge backend over conversational history, the relevant findings are the exact-search numbers and PDX-BOND's no-preprocessing property. A memory store that grows by appending new conversation embeddings daily cannot afford PCA retraining on every ingest, and at personal scale (well under a million vectors) exact search stays feasible, which sidesteps recall tuning entirely. At organizational scale the IVF results apply, and the layout composes with existing indexes rather than replacing them. The broader lesson for persistent AI memory work is that retrieval cost is partly a systems problem: the same algorithm swung from slower than a linear scan to 4.6x faster on the strength of data layout alone.

Read from: full text

User Feedback in Human-LLM Dialogues: A Lens to Understand Users But Noisy as a Learning Signal

Liu, Zhang, Choi 2025, User Feedback in Human-LLM Dialogues, EMNLP 2025, arXiv:2507.23158

The paper asks whether implicit user feedback in chat logs (a user saying “could you label the y-axis?” rather than clicking a thumbs-down) can be detected reliably and then used to improve a model. Working with two real interaction corpora, LMSYS-chat-1M and WildChat, the authors densely annotate 109 conversations (75 from LMSYS, 34 from WildChat), labeling every user turn after the first with a six-way ontology inherited from Don-Yehiya et al. 2024: positive feedback, four negative types (rephrasing, make aware with or without correction, ask for clarification), and no feedback. Inter-annotator agreement is Cohen’s kappa 0.70 for binary and 0.60 for fine-grained labels. They then build a GPT-4o-mini classifier with in-context examples and, in the second half, test whether using the content of negative feedback, not just its polarity, produces better training data: a stronger model regenerates the unsatisfying response conditioned on the user’s complaint, and the result is distilled into 7B models (Vicuna and Mistral) by supervised fine-tuning on 20K conversations.

The descriptive findings are the cleaner half. Feedback is common and skews negative: in later turns it appears in more than half of user utterances, while praise is rare. Refusals explain almost none of it (under 3 percent of sampled responses in either corpus). More troubling for naive harvesting, prompts that elicit positive feedback are slightly more toxic and lower quality than random ones; manual inspection traces this to users praising successful jailbreaks. Their detection prompt roughly doubles prior accuracy on the dense setting (41.6 versus 31.5 percent binary). The training results are mixed and scope-bound: feedback-conditioned regeneration beats regeneration from scratch on MTBench (80 short human-written questions; Vicuna-7b reaches 6.86 versus 6.38), but on WildBench (500 longer, more complex real-user questions) it loses (23.38 versus 27.97 for the same setup), and plain distillation from the stronger model on random conversations does best of all.

For anyone building a personal knowledge backend over conversational history, the annotation study is the useful part: it establishes that later turns are dense with correction signal marking unmet intent, exactly the turns a memory system should weight when inferring what a user actually wants, while the training results warn that this signal cannot be consumed raw. Positive reactions encode bad behavior as well as good, and inferring unspoken intent can erode adherence to well-specified requests. Both lessons rest on two 2023-era public corpora and 7B models, one of which (LMSYS) reflects evaluation play rather than real work, so organizational logs may behave differently.

Read from: full text

Evaluating Very Long-Term Conversational Memory of LLM Agents

Maharana et al. 2024, LoCoMo: Evaluating Very Long-Term Conversational Memory, ACL 2024, arXiv:2402.17753

The paper builds LoCoMo, a dataset of 50 very long two-person conversations, each averaging about 300 turns and 9,200 tokens across up to 35 sessions, roughly nine times the length of MSC, the previous standard. Rather than collect real conversations over months, the authors generate them: two gpt-3.5-turbo agents, each seeded with an expanded persona from MSC and a temporal event graph of up to 25 causally linked life events spanning 6 to 12 months, converse using a memory architecture borrowed from Park et al.'s generative agents (per-session summaries as short-term memory, per-turn observations as long-term memory). Agents can also share web-searched images and react to them. Human annotators then edited about 15 percent of turns and replaced or removed about 19 percent of images to fix long-range inconsistencies. On top of the dataset sit three tasks: question answering across five reasoning types (single-hop, multi-hop, temporal, open-domain, adversarial), event summarization scored with an adapted FactScore, and multimodal dialogue generation.

The headline finding is that every tested configuration sits far below the human QA benchmark of 87.9 F1. GPT-4-turbo with a 4K window is the best base model at 32.1 overall. Extending context helps ordinary recall (gpt-3.5-turbo-16k climbs from 24.1 to 37.8 as its window grows from 4K to 16K) but collapses on adversarial questions, falling to 2.1 F1 against 70.2 for GPT-4-turbo at 4K: given more context, the model hallucinates answers to unanswerable questions and misattributes events to the wrong speaker. Temporal reasoning stays weak everywhere. On event summarization the long-context model actually underperforms the 4K base model (39.9 versus 45.9 FactScore F1), and the common failure modes are missed causal links, hallucinated details, and wrong speaker attributions. All of this is on 50 English, LLM-generated-then-edited conversations, so the numbers describe that synthetic corpus, not real chat logs.

The most transferable result concerns memory representation. RAG over raw dialogue logs helps modestly, but RAG over "observations," per-turn assertions distilled about each speaker's life, is the strongest configuration: top-5 observations reach 41.4 overall F1, and performance degrades as more are retrieved, which the authors read as a signal-to-noise problem. Session summaries retrieve well but answer poorly, suggesting summarization loses the facts QA needs. For a personal knowledge backend over conversational history, that ordering (distilled assertions beat both raw transcripts and summaries, and retrieval precision beats retrieval volume) is a directly usable design finding, and the adversarial collapse is a warning that scale of context is not the same as memory.

Read from: pages 1-10 of 19 (body complete; remainder is references and appendix)

Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs

Malkov & Yashunin, Hierarchical Navigable Small World graphs, TPAMI, arXiv:1603.09320

The paper introduces HNSW, a graph index for approximate K-nearest-neighbor search that needs no auxiliary coarse-search structure. Elements are inserted one at a time; each is assigned a maximum layer drawn from an exponentially decaying distribution, and a proximity graph is built at every layer it occupies. Search starts at the sparse top layer, greedily descends to a local minimum, then drops a layer and repeats, so the structure behaves like a probabilistic skip list with proximity graphs in place of linked lists. The second contribution is a neighbor-selection heuristic: rather than linking a new element to its M closest candidates, it keeps only candidates closer to the new element than to any already-selected neighbor, which recovers a relative neighborhood subgraph and preserves connectivity across cluster boundaries.

The authors argue O(log N) search and O(N log N) construction, with the caveat that the analysis assumes exact Delaunay graphs and constant node degree; empirical support comes from low-dimensional data (d=4 and d=8 random vectors up to 10M elements), and they state that verifying the resilience of the approximation at d=128 would need infeasibly large datasets. Practical settings fall out of the experiments: M between 5 and 48, ground-layer cap Mmax0 = 2M, level normalizer mL = 1/ln(M), efConstruction near 100, and index overhead of roughly 60 to 450 bytes per object. On the ann-benchmarks datasets (1M SIFT, 1.2M GloVe, 2M CoPhIR, 1M DEEP, 60k MNIST) HNSW beats Annoy, FLANN, FALCONN, and VP-tree by wide margins, and on the wiki-8 dataset under Jensen-Shannon divergence it improves on plain NSW by almost three orders of magnitude in distance computations. Against Faiss product quantization on a 200M SIFT subset, HNSW reaches higher recall at faster query times and builds in 42 minutes versus 11 to 12 hours, but needs 64 Gb of RAM against Faiss's 23.5 to 30, and its query-time scaling there visibly deviates from pure logarithm.

For a conversational-memory backend, HNSW is the algorithm inside most current vector stores, so its tradeoffs are the field's tradeoffs. Insertion is truly incremental with no reshuffling, which fits an append-only history; it handles arbitrary metrics, including edit distance on a 1M DNA dataset. Two limits matter at organizational scale: the paper defers element update and deletion to future work, and distributed search is awkward because every query funnels through the top layer, with skip-graph techniques proposed but not implemented. RAM residency of the full graph is the cost of its speed.

Read from: full text

NoLiMa: Long-Context Evaluation Beyond Literal Matching

Modarressi, Deilamsalehy, Dernoncourt, Bui, Rossi, Yoon, Schütze; ICML 2025 (PMLR 267); arXiv:2502.05167

The paper builds a needle-in-a-haystack benchmark in which the question and the hidden fact share almost no words, so a model cannot find the needle by pattern matching and must instead follow a latent association. A needle like “Actually, Yuki lives next to the Semper Opera House” answers the question “Which character has been to Dresden?” only if the model knows the opera house is in Dresden; two-hop variants stretch the chain further (Dresden to Saxony). The authors quantify the gap they are closing with ROUGE precision between question and relevant context: vanilla NIAH scores 0.905 R-1, NoLiMa scores 0.069. The benchmark uses 58 question-needle pairs built from 5 needle groups, haystacks assembled from snippets of 10 open-license books and filtered (with Llama 3.3 70B plus manual review) to remove accidental distractors and false answers, and 26 needle placements per length, giving 7,540 tests per context length.

Across 13 models claiming at least 128K context, short-context performance is strong (most base scores above 84 percent) but decays fast. At 32K tokens, 11 of 13 fall below half their base score; GPT-4o, the best performer, drops from 99.3 to 69.7. Defining effective length as the longest context holding 85 percent of the base score, most models sit at 2K or below; GPT-4o reaches 8K against a claimed 128K. The same Llama 3.1 70B that reaches 32K effective length on RULER manages 2K here, which isolates literal matching as the crutch. Two-hop questions and inverted fact order (keyword before character name) degrade further, and an aligned-depth analysis of the last 2K tokens shows the drop tracks total context length rather than needle position, implicating attention capacity rather than position encoding. CoT prompting helps (two-hop at 16K goes from 42.7 to 56.7 for Llama 3.3 70B) but does not fix it; on a hard subset, GPT-o1 and DeepSeek-R1 still fall below 50 percent of base at 32K. Restoring literal matches via multiple choice lifts two-hop 32K from 25.9 to 87.2, while a single distractor sentence sharing words with the question cuts GPT-4o’s effective length to 1K.

For a conversational-history backend, the message is that dumping months of transcripts into context and asking associative questions (“when did I decide X”) will fail well before the advertised window, especially when the query is phrased differently than the original conversation, which is the normal case. Retrieval must do semantic bridging before the model sees anything, keep contexts short, and avoid surfacing lexically similar but irrelevant passages, since those actively mislead. The scope caveat: results come from synthetic single-fact needles in book-text haystacks up to 32K (128K for two models), not from real dialogue data.

Read from: full text

Industry-scale Knowledge Graphs: Lessons and Challenges

Noy, Gao, Jain, Narayanan, Patterson, Taylor; ACM Queue 17(2) / CACM 62(8), 2019; DOI 10.1145/3331166

This is an experience paper, not an empirical study: practitioners from Google, Microsoft, Facebook, eBay, and IBM (expanding on a 2018 ISWC panel) each describe their production knowledge graph, then pool the challenges that recur across all five. It proceeds company by company through design decisions, then through shared open problems: entity disambiguation, type membership, changing knowledge, extraction from unstructured sources, and operating at scale.

The scale numbers frame everything. Google holds roughly 1 billion entities and 70 billion assertions; Microsoft's Bing graph about 2 billion entities and 55 billion facts; Facebook about 50 million entities and 500 million assertions; eBay expects 100 million products and over a billion triples; IBM's framework is proven past 100 million documents and 5 billion relationships. All five use strongly typed entities and relations under a schema or ontology; correctness at this size depends on machinery rather than curation. Bing's single entry for the actor Will Smith is assembled from 108,000 facts across 41 websites, against 200 Will Smiths in Wikipedia alone, which is why the authors name disambiguation the top industry challenge despite years of research. The paper offers mechanisms rather than solutions, and mostly self-reported ones with no benchmarks. Google bootstraps its schema from a small set of low-level structures plus metatypes (types as instances of types) to validate invariants at scale, such as painters predating their paintings. Facebook keeps provenance on every assertion, defines correctness as always being able to explain why an assertion was made, and rebuilds the whole graph from sources daily through an idempotent build. eBay funnels all writes through a replicated log feeding a low-latency document store and a separate analytical graph store. IBM uses polymorphic stores, treats source evidence as a primitive, and defers entity resolution to query time so that context can settle ambiguous names.

For a personal knowledge backend built over conversational history, the transferable material is architectural. Provenance-first assertions, identity managed separately from surface forms, late-binding disambiguation, and the log-plus-specialized-stores pattern all apply directly to conflicting, evolving conversational facts, and IBM's layering of general, domain, and private customer knowledge is essentially the personal-versus-shared split an organizational rollout would need. The authors also flag personalized on-device graphs, small enough to ship to a phone, as an open direction they invite research on. As a representation reference the paper shows the dominant industrial pattern, typed property graphs with schema-level reasoning, but it describes no query languages, storage internals, or evaluation, so it orients rather than specifies.

Read from: full text

ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory

Ouyang et al. 2026, ICLR 2026, arXiv:2509.25140

ReasoningBank is a memory framework for LLM agents that face a stream of tasks with no ground-truth feedback. Instead of storing raw trajectories (Synapse) or success-only workflows (AWM), it distills each finished episode into structured memory items, each with a title, a one-sentence description, and content holding the abstracted strategy or pitfall. An LLM-as-a-judge labels each trajectory success or failure without reference answers, and both outcomes feed extraction: successes yield validated strategies, failures yield preventative lessons. At each new task the agent retrieves the top-k items by embedding similarity and injects them into its system instruction; new items are consolidated back by simple addition, closing the loop. The authors also propose memory-aware test-time scaling (MaTTS), which spends extra compute per task (k parallel rollouts with self-contrast, or k sequential self-refinements) and aggregates across the redundant attempts to curate memory, rather than converting each rollout independently.

On WebArena (684 tasks, Map excluded) with Gemini-2.5-flash, ReasoningBank lifts overall success rate from 40.5 (no memory) to 48.8, and MaTTS at k=5 reaches 51.8; gains of +7.2 and +4.6 hold for Gemini-2.5-pro and Claude-3.7-sonnet. On SWE-Bench-Verified, resolve rate rises from 34.2 to 38.8 (flash) and 54.0 to 57.4 (pro), and Mind2Web improves across cross-task, cross-website, and cross-domain splits. Steps per task drop as well, up to 1.4 versus no memory, with the largest cuts on successful runs (2.1 fewer steps on Shopping, a 26.9 percent reduction), so memory shortens good paths rather than truncating bad ones. Ablations on WebArena-Shopping with flash show failures matter: success-only extraction gives 46.5, adding failures 49.7, while AWM degrades from 44.4 to 42.2 when fed failures. The judge itself is only 72.7 percent accurate there, yet simulated accuracy from 70 to 90 percent barely moves the results. A case study shows memory items maturing from procedural rules into compositional checking strategies over the task stream.

For a personal knowledge backend over conversational history, the transferable finding is representational: distilled, self-contained strategy units with title, description, and content beat both raw transcripts and success-only procedures, and failure records carry real signal if the schema is built to hold lessons rather than replays. The robustness to a noisy self-judge suggests curation can run without human labels, which is what organizational scale requires. Scope limits apply: benchmarks are web browsing and repository-level software fixes, streams are hundreds of tasks, consolidation is pure addition with no pruning or deduplication, so behavior over years of accumulation is untested.

Read from: pages 1-12 of 30

Unifying Large Language Models and Knowledge Graphs: A Roadmap

Pan et al. 2024, IEEE TKDE, arXiv:2306.08302

This is a survey and roadmap, not an experimental paper. The authors organize research on combining LLMs and knowledge graphs into three frameworks: KG-enhanced LLMs (graphs injected during pre-training, at inference, or used to probe and interpret models), LLM-augmented KGs (language models applied to KG embedding, completion, construction, graph-to-text generation, and question answering), and Synergized LLMs + KGs, where the two are trained or operated jointly for representation and reasoning. Under each framework they build a fine-grained taxonomy and tabulate representative methods, roughly a hundred systems spanning 2018 to 2023, from ERNIE and K-BERT through RAG, KEPLER, DRAGON, StructGPT, and Think-on-Graph, each tagged by technique and by the kind of graph or model involved.

Because it synthesizes rather than measures, its findings are the trade-offs it documents. Pre-training injection yields the strongest downstream performance on knowledge-intensive tasks but never permits knowledge updates without retraining; retrieval at inference handles frequently changing open-domain facts but the underlying model was never trained to exploit them, so results can be sub-optimal. For KG completion, encoder-style methods (KG-BERT and successors) must score every candidate entity and cannot handle unseen entities, while generator-style methods generalize and skip ranking but may produce entities that do not exist in the graph. The probing work it collects is sobering: LAMA-style cloze tests show LLMs recall popular facts, but a Wikidata study of low-frequency entities found scaling fails to appreciably improve memorization of tail facts, and a causal analysis (Shaobo et al. 2022) found models complete facts from positionally close words rather than knowledge-bearing ones.

For a personal knowledge backend over conversational history, two threads matter most. First, the LLM-augmented KG construction pipeline (entity discovery, coreference resolution, relation extraction, plus end-to-end systems like Grapher and PiVE, which uses a small T5 to iteratively correct a larger model's extracted triples) is exactly the machinery for distilling structured, queryable memory from raw transcripts, and the survey's own trade-off analysis argues for inference-time retrieval over parameter injection whenever the store must stay editable. Second, the agent-style reasoning line (StructGPT, Think-on-Graph) shows LLMs traversing a graph step by step without retraining, giving interpretable answers that cite paths, which is the pattern an organizational memory would need for auditability. The future-directions section flags the open problems such a system inherits: hallucination detection against the graph, the ripple effects of editing one fact, and injecting knowledge into black-box models where prompts are bounded by context length.

Read from: pages 1-24 of 28

Generative Agents: Interactive Simulacra of Human Behavior

Park, O'Brien, Cai, Morris, Liang, and Bernstein, UIST 2023, arXiv:2304.03442

The paper builds twenty-five LLM-driven agents, each seeded with one paragraph of natural language identity, and runs them in Smallville, a Sims-like sandbox town, to test whether an architecture wrapped around a language model (gpt-3.5-turbo; GPT-4 was invitation-only at the time) can produce believable long-horizon behavior. The architecture has three parts. A memory stream records every experience as a timestamped natural language object; retrieval scores each record as an equally weighted sum of recency (exponential decay, factor 0.995 per game hour since last access), importance (the LLM rates poignancy 1 to 10 at creation), and relevance (cosine similarity of embeddings against a query), min-max normalized. Reflection fires when summed importance of recent events passes a threshold of 150, roughly two or three times per agent day: the model proposes three salient questions from the 100 most recent records, retrieves against each, and writes cited higher-level inferences back into the stream, forming trees whose leaves are raw observations. Planning drafts a day in five to eight chunks, then recursively decomposes to hour and then 5 to 15 minute actions; plans and reflections re-enter the stream so retrieval mixes all three.

In a controlled evaluation, 100 Prolific raters ranked interview responses from the full architecture, three ablations, and a crowdworker-authored baseline. The full system scored highest (TrueSkill mu 29.89) and each removed component cost believability, down to mu 21.21 with no memory at all; the fully ablated condition, which stands in for prior first-order prompting work, trailed the full system by a standardized effect of d = 8.16, and it was statistically indistinguishable from the human crowdworkers. In a two-game-day end-to-end run, knowledge of a party spread from one agent to thirteen (4% to 52%) and a mayoral candidacy from one to eight, with zero hallucinated claims of knowing; network density rose from 0.167 to 0.74 with 1.3% hallucinated acquaintance claims; five of twelve invitees attended the party. All of this holds only at this scale: 25 agents, two simulated days, one hand-built town, at a cost of thousands of dollars in tokens.

For a personal knowledge backend, the transferable finding is that a flat log plus scored retrieval is not enough; the ablations show believable use of history required the synthesized layer too, and the failure catalog (missed retrievals, embellished recall, one agent conflating a neighbor named Adam Smith with the economist) is a checklist of what such a backend must guard against. This paper fixed the memory-stream, reflection, retrieval-triple vocabulary that most subsequent persistent-memory work inherits. The authors also flag memory hacking, planted false events, as an open robustness problem, which becomes an access-control question at organizational scale.

Read from: full text

Beyond the Context Window: A Cost-Performance Analysis of Fact-Based Memory vs. Long-Context LLMs for Persistent Agents

Pollertlam & Kornsuwannawit (Bricks Technology), arXiv preprint, 2026, arXiv:2603.04814

The paper asks a deployment question: for an assistant that must remember a user across sessions, is it better to resend the whole conversation history to a long-context model every turn, or to extract facts once into a memory store and retrieve only what each query needs? The authors build the memory side on the open-source Mem0 pipeline (GPT-5-nano extracts atomic flat-typed facts, text-embedding-3-small embeds them into pgvector, top-20 retrieval feeds a GPT-5-mini reader) and compare it against long-context inference with GPT-5-mini and GPT-OSS-120B on three benchmarks: LongMemEval (500 conversations, about 101,600 tokens each), LoCoMo (10 dialogues, 1,986 questions), and PersonaMem v2 (222 conversations, 589 questions). Accuracy is scored by GPT-5-mini as judge with a three-vote majority; cost is modeled from actual API spend with a 90 percent prompt-caching discount applied to the long-context side from turn two onward.

Long-context GPT-5-mini wins decisively on factual recall: 92.85 percent versus 57.68 on LoCoMo and 82.40 versus 49.00 on LongMemEval, gaps of 35.2 and 33.4 points. On PersonaMem v2 the gap shrinks to 7.3 points, and the memory system edges out long-context GPT-OSS-120B (62.48 versus 60.50), because persona attributes are exactly the stable facts flat extraction captures well. The cost picture inverts with use. Extraction compresses a 101,600-token history to about 2,909 retrieved tokens (roughly 35:1), a one-time write of $0.0430 per user, after which each query costs about $0.0013; the cached long-context turn still scales with context length. At 100k tokens the memory system becomes cheaper after about ten turns and saves 26 percent by turn twenty; the break-even falls from 13 turns at 30k to 9 at 500k. An error analysis names what compression loses: precise temporal markers, implicit coreferences, and updates that never propagate (a “twice a week” fact surviving a later revision to three). All accuracy results are scoped to Mem0’s flat extraction; the authors caution they do not generalize to structured memory architectures like Zep or temporal semantic memory.

For a personal knowledge backend over conversational history, this paper supplies the economic argument and its price: at organizational scale, per-query cost stays near-flat only if you accept extraction loss, and the three failure modes are a concrete checklist of what a fact schema must handle (timestamps, role resolution, update propagation). The finding that answer-model capability, not context length, limited GPT-OSS-120B is a useful caution against attributing retrieval failures to the memory layer. The static-snapshot protocol, with no incremental updates during querying, marks the gap between these benchmarks and a live system.

Read from: full text

Zep: A Temporal Knowledge Graph Architecture for Agent Memory

Rasmussen, Paliychuk, Beauvais, Ryan, Chalef (Zep AI), arXiv preprint, 2025, arXiv:2501.13956

The paper describes Zep, a commercial memory service for LLM agents, and its core engine Graphiti, a temporally aware knowledge graph built continuously from conversation. Incoming messages become episode nodes, a non-lossy raw store; an LLM pipeline (gpt-4o-mini, with a reflection step to curb hallucination) extracts entities and facts from each message plus the last four for context, embeds them in 1024 dimensions, and resolves duplicates against existing nodes through hybrid search and an LLM judgment. Above the entity layer, communities of strongly connected entities carry map-reduce style summaries, maintained incrementally by label propagation rather than Leiden so the graph can extend without full recomputation. The distinctive piece is the bi-temporal model: every fact edge carries four timestamps, two for when the system learned and expired it and two for when it held true in the world, and new edges that contradict old ones invalidate them rather than overwrite them. Retrieval runs cosine similarity, BM25, and graph breadth-first search in parallel, reranks (RRF, MMR, mention frequency, node distance, or cross-encoder), and formats roughly 1.6k tokens of facts with validity ranges plus entity summaries.

On MemGPT's Deep Memory Retrieval benchmark (500 conversations of only about 60 messages each), Zep scores 94.8% with gpt-4-turbo against MemGPT's 93.4%, and 98.2% with gpt-4o-mini; the authors themselves note that simply stuffing the full conversation into context scores 94.4% and 98.0%, so DMR barely discriminates. The stronger evidence is LongMemEval, with conversations averaging 115k tokens: Zep beats the full-context baseline by 15.2% with gpt-4o-mini (63.8% vs 55.4%) and 18.5% with gpt-4o (71.2% vs 60.2%), while cutting latency about 90% (2.58s vs 28.9s with gpt-4o). Gains concentrate in temporal reasoning, multi-session synthesis, and preference questions; single-session-assistant questions actually drop 17.7% under gpt-4o, an admitted failure. MemGPT could not be run on LongMemEval at all.

For a personal memory backend over conversational history, this is close to a reference design: episodic raw storage under a derived semantic layer, entity resolution as the hard ongoing problem, and explicit fact validity intervals as the answer to knowledge that changes, which flat retrieval handles badly. The retrieval numbers also argue that a graph layer earns its cost mainly at scale; below context-window size it buys little accuracy. The paper doubles as a critique of the field's evaluation habits: DMR is too small and too fact-retrieval shaped, and no benchmark yet tests fusing conversation with structured business data, which is exactly the organizational-scale case. All results depend on chat-style corpora and OpenAI models.

Read from: full text

Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control

Rezazadeh, Li, Lou, Zhao, Wei, Bao (Accenture Center for Advanced AI), arXiv preprint (under review), 2025, arXiv:2505.18279

The paper asks how a system serving many users with many LLM agents can share memory across user boundaries without leaking anything a given user is not entitled to see. Its answer is a framework, not a learned model: permissions are encoded as two time-varying bipartite graphs (user-to-agent and agent-to-resource), and memory is split into a private tier scoped to the originating user and a shared tier open to cross-user retrieval. Every memory fragment carries immutable provenance: creation time, contributing user, contributing agents, and resources touched. A fragment is readable in a given user-agent context only if its contributing agents fall within the user's current agent permissions and its source resources fall within the agent's current resource permissions, so revoking an edge in the graph retroactively hides fragments that depended on it. Read and write policies, configurable globally, per agent, or per user, filter and transform fragments on the way in and out; the implementation runs everything on gpt-4o with text-embedding-3-large retrieval, a coordinator that routes subqueries to specialist agents, and an aggregator that composes the final answer.

Three scenarios carry the evaluation, all at small scale. On MultiHop-RAG (609 news articles across six domains, 2,556 multi-hop questions, five users, six domain agents), fully shared memory holds accuracy above 0.90 while cutting external resource calls by up to 61 percent at 50 percent query overlap and 59 percent at 75 percent overlap, relative to per-user isolated memory. A second scenario uses 200 synthetic business queries under asymmetric roles (only a Strategy Director sees all four agents); with no ground truth available it reports only that partial sharing reduces resource calls for every role. The third uses SciQAG science QA (five categories, five users, 100 queries) with permissions granted through step t4 and revoked through t8: accuracy rises and falls with the access graph, resource calls decline over time as memory substitutes for retrieval, and logged access matrices show no fragment crossing a permission boundary. The authors concede the settings are simulated, the user counts modest, and that LLM-based policy enforcement can still occasionally breach policy.

For a personal knowledge backend built over one person's conversational history, the relevant machinery is the provenance-stamped fragment plus retrospective permission check: it is exactly the primitive needed if a single-user memory tree ever grows organizational tenants, since access can change after fragments exist without rewriting the store. The efficiency result also matters, as it quantifies memory's role as a cache that displaces repeated retrieval. Within the field, the paper marks the point where LLM memory work imported access-control formalism (RBAC and ABAC lineage) rather than assuming one global store, though it establishes a formulation and feasibility evidence, not a benchmark others can yet be measured against.

Read from: full text

From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs

Rezazadeh, Li, Wei, Bao (Accenture Center for Advanced AI), ICLR 2025, arXiv:2410.14052

MemTree is an online memory system that stores an LLM agent's accumulating history as a dynamically grown tree rather than a flat embedding table. Each node holds aggregated text, an embedding of that text, and links to parent and children; depth corresponds to abstraction, with general summaries near the root and specifics at the leaves. New information is inserted by traversing from the root: at each node, cosine similarity against the children is compared to a depth-adaptive threshold (theta grows exponentially with depth, so deeper, more specific nodes demand closer matches). A sufficiently similar child is descended into; otherwise a new leaf is attached and traversal stops. Every ancestor on the path then has its summary rewritten by an LLM call to fold in the new content, and its embedding refreshed. Insertion is O(log N), and the authors connect the scheme to online top-down hierarchical clustering, proving a beta/3 approximation of the optimal Moseley-Wang revenue under a well-separatedness assumption. Retrieval collapses the tree: the query embedding is compared against every node at once, filtered by a threshold, and the top-k nodes are returned.

The evaluation, run with GPT-4o plus text-embedding-3-large and separately Llama-3.1-70B, covers four benchmarks. On the 15-round Multi-Session Chat sessions, simply feeding GPT-4o the full history wins (95.6 percent accuracy) since each dialogue is under a thousand tokens; among memory systems MemTree edges MemoryStream (84.8 vs 84.4) and beats MemGPT (70.4). The interesting result comes from MSC-E, the authors' 70-session extension to 200 dialogue rounds: there MemTree (82.5 overall) beats both full-history prompting (78.0) and MemoryStream (80.7), and holds accuracy steady across evidence positions where the naive baseline degrades. On QuALITY single-document QA with Llama-3.1-70B, MemTree (59.8) roughly matches the offline RAPTOR (59.0) despite building its index incrementally. On MultiHop RAG (609 news articles, 2,556 questions), MemTree reaches 80.5 overall, within 0.5 points of RAPTOR and above GraphRAG (78.3), and on temporal queries (68.4) it beats every baseline including human-annotated evidence. The learned tree over those 609 documents has 3,164 nodes, depth 13, branching factor 2.1. Per-item insertion averages 10 seconds; cumulative cost runs about 1.4x the offline methods.

For a personal knowledge backend over conversational history, the central lesson is that the online-versus-offline distinction matters more than the specific structure: RAPTOR and GraphRAG require full corpus rebuilds to absorb new material, while MemTree pays a modest ongoing tax to stay current, which is the right trade for a system ingesting conversations continuously. Two caveats travel with the results: parent summaries are LLM-rewritten on every insertion, so ancestor content drifts with the insertion order and inherently favors recent material (the authors show accuracy rising from 82.1 to 84.2 for late-positioned evidence), and the corpora tested top out at hundreds of documents, well short of organizational scale.

Read from: pages 1-12 of 34 (full main text; remainder is references and appendices)

Knowledge Graph Embedding for Link Prediction: A Comparative Analysis

Rossi, Firmani, Matinata, Merialdo, and Barbosa, ACM TKDD, 2021, arXiv:2002.00819

The paper retrains and re-evaluates 16 embedding-based link prediction models from scratch, spanning three families (tensor decomposition, geometric, and deep learning), against AnyBURL, a rule-mining baseline, on the five standard benchmarks: FB15k, WN18, FB15k-237, WN18RR, and YAGO3-10. Beyond the usual global metrics (Hits@K, mean rank, mean reciprocal rank), the authors define per-fact structural features of the training graph and correlate each with prediction accuracy: the number of "peers" (alternative valid answers seen in training), a TF-IDF style Relational Path Support score for paths up to three hops, relation properties such as symmetry and transitivity, and the degree of the original reified node for facts derived from FreeBase hyperedges.

Three findings carry the paper. First, the rule-based baseline holds its own: AnyBURL ranks at or near the top on almost every dataset and metric while learning its rules in 100 to 1,000 seconds, against roughly 1 to 300 hours of training for the embedding models; among those, only ComplEx with N3 regularization is consistently comparable. Second, graph structure largely determines difficulty: more source peers help (they are worked examples of the answer), more target peers hurt, and higher path support helps nearly every model. On FB15k and WN18 the leakage from inverse relations is so severe that the path-support score alone, used as a scoring function with two-hop paths on a 512-fact sample, reaches 93.55 percent H@1 on WN18. Third, evaluation plumbing matters: the policy for breaking score ties, rarely even reported, produces wildly incomparable numbers. CapsE scores 73.01 H@1 on FB15k under the min policy and 1.93 under average, and a perfect 100 versus 0 on YAGO3-10; the authors trace the ties to saturating activations (ReLU, and sigmoid in its near-flat regions) and show a leaky-ReLU variant mostly closes the gap.

For a personal knowledge backend over conversational history, the transferable lessons are less about any single model than about method. A graph distilled from one person's conversations will be sparse and skewed, with few peers and thin path support per fact, which is precisely the regime where every tested model performs worst; the cheap, interpretable rule-based approach is the sensible starting point before any embedding machinery. And the tie-policy result is a standing caution for persistent-memory evaluation generally: retrieval benchmarks can flatter or bury a system through unstated protocol details. Scope limit throughout: all results come from graphs of 14k to 123k entities sampled from FreeBase, WordNet, and YAGO, well below organizational scale.

Read from: full text

Bitemporal Property Graphs to Organize Evolving Systems

Rost, Fritzsche, Schons, Zimmer, Gawlick, Rahm; arXiv preprint, 2021; arXiv:2111.13499; a summary report on a one-year Oracle and University of Leipzig cooperation.

This paper reports the outcomes of a joint project between the University of Leipzig and Oracle on organizing relationships within multi-dimensional time-series data, with IoT sensor networks (an aircraft's instrumented components is the running example) as the motivating case. Rather than a single deep result, it presents four connected artifacts: TPGM+, a bitemporal property graph model; T-PGQL, a temporal extension of Oracle's PGQL query language; CGN, a continuous event-detection mechanism; and BiTeGra, a prototype database that stores the graphs in a relational system. The paper walks through each in turn, with the query language treated in the most detail.

The core of TPGM+ is bitemporality carried down to the property level. Every vertex, edge, and property value gets two close-open time intervals, valid-time (when the fact held in the world) and transaction-time (when the database learned it), plus five integrity constraints covering uniqueness, referential integrity of edges and of properties, constant edge endpoints, and constant labels. The property-level intervals are the delta over the authors' earlier TPGM, which had to copy an entire element whenever one property changed. T-PGQL exposes the intervals through dot-notation accessors (VAL_FROM, TX_TIME, and so on), SQL:2011 period predicates (CONTAINS, OVERLAPS, PRECEDES), a FOR TX_TIME clause for snapshot and range time travel with AS OF, FROM TO, BETWEEN AND, and ALL variants, and FIRST/LAST aggregates. CGN adapts Oracle's Continuous Query Notification to graphs, distinguishing notifications on any change to matched elements from notifications only when the query result changes. BiTeGra maps graphs to bitemporal relational tables and weighs schema choices explicitly: one big vertex-and-edge table pair versus a table per label, and properties as columns versus a key-value property table, with hybrid schemas (HyVE, HyPe) that record the per-label choice in metadata. The paper presents no benchmarks or evaluation; the language covers retrieval only, not data manipulation, and the system is a prototype.

For a personal knowledge backend over conversational history, the transferable idea is the two-clock design: separating when something was true from when the system recorded it is exactly what distinguishes "he changed jobs in May" from "I learned this in July," and doing it per property rather than per entity avoids duplicating a whole record when one fact changes. The FOR TX_TIME AS OF construction is a ready-made vocabulary for asking what the system believed at a past moment, useful for auditing an agent's evolving beliefs. The schema-mapping section is also a compact survey of how property graphs sit on relational storage, though the absence of any performance data means the design tradeoffs are argued, not measured.

Read from: full text

LLM-based Multi-Agent Blackboard System for Information Discovery in Data Science

Salemi et al. 2026, LLM-Based Multi-Agent Blackboard System, arXiv:2510.01285

The paper revives the classic blackboard architecture (Hearsay-II lineage, Erman et al. 1980) as a communication paradigm for LLM multi-agent systems, targeting data discovery: answering data science questions when the relevant files sit unidentified inside a large, heterogeneous data lake. Instead of a master agent assigning subtasks to workers it must already understand, the main agent posts a request describing what it needs to a shared blackboard. Helper agents, each responsible for one cluster of the data lake plus one web search agent, monitor the board and volunteer responses only when they judge themselves capable. The data lake is partitioned offline by giving file names to Gemini 2.5 Pro for clustering; each file agent then studies its cluster's structure in advance. The main agent runs a ReAct loop with five actions (plan, reason, execute code, request help, answer) capped at 10 steps, and its final output is a Python program that loads the right files and computes the answer.

Across KramaBench plus discovery-augmented versions of DSBench and DA-Code, the blackboard beats DS-GRU (all files in context), RAG over the lake (E5-large, top 5 files), and a master-slave variant that is otherwise identical, with 13 to 57 percent relative end-to-end gains over the best baseline. The pattern holds across four backbones (Gemini 2.5 Pro and Flash, Claude 4 Opus, Qwen3-Coder 30B): with Gemini 2.5 Pro, macro average rises from 24.0 (master-slave) to 28.5. File discovery F1 reaches 0.561 versus 0.513 for master-slave and 0.247 for RAG, again Gemini 2.5 Pro only. The advantage grows with lake size: the two largest lakes, DA-Code (145 files) and KramaBench Astronomy (1,556 files), show 70 and 211 percent relative gains over master-slave. Costs are real: roughly 2.3 times RAG's per-question spend, though runtime stays comparable (132 to 145 seconds across all three systems on a 50-question sample). Content-embedding clustering outperforms filename clustering, and more clusters generally help.

For a personal knowledge backend over conversational history, the transferable finding is architectural: routing by self-selection beats routing by a central index of who knows what, precisely because a coordinator's model of each partition goes stale. Partition the corpus, let each partition's agent pre-digest its holdings offline, broadcast queries, and let holders volunteer. That pattern sidesteps the retrieval weakness the authors document (embedding retrieval collapses to 0.247 F1 on this heterogeneous data) and scales in the direction an organization's archive grows. The caveats travel with it: results cover data lakes of at most 1,556 files, tabular and scientific domains, and a fixed two-level hierarchy.

Read from: pages 1-14 of 44 (full main text; remainder is references and appendices)

RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

Sarthi, Abdullah, Tuli, Khanna, Goldie, Manning; ICLR 2024; arXiv:2401.18059

RAPTOR replaces the flat chunk store of standard retrieval augmentation with a summary tree built from the bottom up. The corpus is split into 100-token chunks (whole sentences preserved), embedded with SBERT, and then soft-clustered with Gaussian Mixture Models over UMAP-reduced embeddings; cluster count is chosen by BIC, and a chunk can join more than one cluster. Each cluster is summarized by gpt-3.5-turbo, the summaries are re-embedded, and the cycle repeats until clustering is no longer feasible, yielding a tree whose leaves are raw text and whose upper nodes are progressively broader abstractions. Because clustering runs on embeddings rather than document position, distant but related passages can share a parent, which is what separates RAPTOR from adjacency-based hierarchies like LlamaIndex or the recursive summarization of Wu et al. (2021). Construction cost scales linearly in tokens. At query time, the authors' preferred strategy simply collapses the tree: every node at every layer competes in one cosine-similarity search, and nodes are added until a token budget is reached (2,000 tokens, roughly the top 20 nodes, chosen after collapsed tree beat layer-by-layer traversal on 20 QASPER stories).

In controlled comparisons with UnifiedQA-3B as reader, adding the tree improved every retriever tested (SBERT, BM25, DPR) on all three benchmarks: NarrativeQA (1,572 book and movie transcripts), QASPER (5,049 questions over 1,585 NLP papers), and QuALITY (multiple-choice over passages averaging 5,000 tokens). With stronger readers the gains held: on QASPER, RAPTOR reached F-1 of 53.1 with GPT-3 and 55.7 with GPT-4, beating DPR by 1.8 to 4.5 points and edging past long-context models like CoLT5 XL (53.9). The headline result is QuALITY with GPT-4: 82.6 percent accuracy against a prior best of 62.3, and a 21.5-point lead over CoLISA on the hard subset. All corpora are single long documents or paper collections, not open-domain web scale, and an annotation study found about 4 percent of generated summaries contained minor hallucinations, though these did not propagate upward or measurably hurt QA. A layer ablation on one QuALITY story showed full-tree search (73.68) beating any single layer (about 58), evidence that the abstraction hierarchy itself, not just better chunking, does the work.

For a memory backend over conversational history, the transferable idea is that recall should span abstraction levels: some queries need the raw utterance, others need the season-of-work summary, and a collapsed search over both lets the query pick its own granularity. RAPTOR is also a batch-built, static index; conversations accrete, so an incremental variant of its cluster-summarize loop is the open engineering question, and its 4 percent summary hallucination rate is a caution for any system whose upper layers become the record people trust.

Read from: pages 1-12 of 23 (full main text; references and appendices skipped)

Developing Time-Oriented Database Applications in SQL

Richard T. Snodgrass, Morgan Kaufmann (Series in Data Management Systems), 1999, book, 528 pp. in this PDF

This is an orientation to a book, not a summary of a paper, and it draws on the parts that state the argument: the preface, chapter 1, and the opening of the bitemporal chapter. Snodgrass translates roughly 1600 research papers on temporal databases into guidance for people writing ordinary SQL. The preface opens with a development story that reads like a confession. A team adds one DATE column, finds the primary key no longer holds, adds a second DATE column and inherits a crop of off by one bugs where a comparison should have been less than or equal, or where a query needs a "+ 1 DAY" correction, then learns that two reports run days apart cannot be reconciled at all. The claim is that this is a category error, not sloppiness. The teaching runs through case studies from real installations: oil field records, cattle location data, a Danish mortgage bank.

The organizing scheme is three orthogonal triples. The temporal data types are instant, interval, and period. The kinds of time are user defined time (an uninterpreted value), valid time (when a fact was true in the modeled world), and transaction time (when a fact was stored). The kinds of statement are current, sequenced, and nonsequenced, and that split applies to queries, modifications, views, and constraints. Snodgrass is blunt about tool support as of 1999. SQL-92 offers DATE, TIME, TIMESTAMP, and INTERVAL but nothing for valid or transaction time; sequenced statements, which he calls the most useful case, get no support whatsoever; SQL3's Part 7, SQL/Temporal, which adds a PERIOD constructor, was not expected to reach ballot before the next millennium. The rest falls to the developer, with code fragments tested against DB2 UDB, Informix, Access, SQL Server, Sybase, Oracle8, and UniSQL, none fully compliant.

The bitemporal chapter is what bears on memory systems. Nykredit, a Danish mortgage bank carrying nine million loans against eight million customers and seven million properties, tracks both times on its property ownership table, giving each row six columns: two foreign keys, VT_Begin and VT_End, TT_Start and TT_Stop. The payoff is a repair procedure. When someone reports bad data, staff find when the error was stored, roll the table back to that transaction time, read the valid-time history as it then stood, and fix it; because transaction time is append only, the fix is itself logged. That is the operation a conversational memory backend needs when a stored belief turns out to be wrong. The costs are equally plain: a sequenced primary key cannot be expressed as a PRIMARY KEY constraint and needs an assertion, modifications multiply into nine forms, and the discipline lives in the application, not the engine.

Read from: pages 17-33 and 301-308 of 528

Cognitive Architectures for Language Agents

Sumers, Yao, Narasimhan, Griffiths 2024, Cognitive Architectures for Language Agents, TMLR, arXiv:2309.02427

This is a conceptual paper, not an experimental one. The authors argue that LLM-based agents recapitulate a sixty-year lineage running from Post's production systems through Markov algorithms to cognitive architectures like Soar, with an LLM playing the role of a probabilistic production system: where a production rewrites string XYZ to XWZ, an LLM defines a distribution over completions of a prompt. On that analogy they build CoALA, a framework that describes any language agent along three dimensions: memory (working memory plus long-term episodic, semantic, and procedural stores), an action space split into external grounding actions and internal retrieval, reasoning, and learning actions, and a decision cycle that plans (propose, evaluate, select) and then executes one grounding or learning action per loop.

The paper's product is the organizing power of that scheme rather than any benchmark number. Table 2 casts prominent agents into the framework and exposes real structural differences: SayCan is evaluation-only over a fixed set of 551 robot skills with no internal actions; ReAct adds reasoning but has no long-term memory or learning; Voyager exercises all four action types, learning by writing code skills into procedural memory; Generative Agents write LLM reflections over episodic memory into semantic memory; Tree of Thoughts has elaborate propose-evaluate-select decision-making but no persistence at all. From the gaps the survey exposes, the authors derive directions: learned retrieval procedures are essentially unstudied, deletion and modification of memory (unlearning) is neglected, and almost no agent treats learning as a deliberately chosen action rather than a fixed schedule. All of this is a reading of the 2022-2023 agent literature, so its claims about what is understudied are claims about that snapshot, not measurements.

For a personal knowledge backend built over conversational history, the useful contribution is the memory typology and its access rules. The paper's worked example is close to that exact problem: a retail assistant with read-only semantic memory (the inventory), read-write episodic memory (past interactions), and episodes summarized by the LLM before storage rather than logged raw. Reflection, in the Generative Agents sense, is the mechanism that turns accumulated episodes into reusable semantic facts, which is the step a conversation archive needs to scale beyond retrieval. The paper also gives the field a shared vocabulary; most persistent-memory systems since can be read as instantiations of its episodic-semantic-procedural split. Its caveat is worth keeping too: writing to procedural memory (an agent's own code or prompts) is flagged as the riskiest form of learning, and longer context windows may shift how much long-term memory matters at all.

Read from: full text

Toward a Theory of Hierarchical Memory for Language Agents

Talebirad, Parsaee, Szepesvari, Nadiri, and Zaiane; ICLR 2026; arXiv:2603.21564

This is a theory paper, not an empirical one. The authors observe that a wide range of long-context and agentic memory systems, from document-hierarchy retrievers like RAPTOR and GraphRAG to conversational memories like xMemory and H-MEM to agent execution-trace managers like MemoBrain and StackPlanner, all implement the same underlying pipeline, and they formalize it as three operators. Extraction (alpha) maps raw data to a graph of atomic information units; coarsening (C), a pair of a grouping function pi and a representative function rho, partitions units and assigns each group a compressed representative, iterated to build a multi-level hierarchy; traversal (tau) takes a query and a token budget and returns a subset of atoms. They prove simple information-theoretic facts about this structure: each deterministic, lossy coarsening level strictly decreases entropy, and by the data processing inequality mutual information with any query is monotonically nonincreasing up the hierarchy, so the atom layer sets the information ceiling for everything above it.

The central contribution is the self-sufficiency spectrum and the coarsening-traversal coupling. Self-sufficiency, SS = I(G; rho(G)) / H(G), measures how much of a group's information its representative preserves, from LLM summaries near 1 to bare category labels near 0. The coupling claim is that this value dictates viable retrieval: self-sufficient representatives support collapsed search over all levels at once (RAPTOR's pattern), while referential ones require top-down refinement (H-MEM, xMemory); mismatching the two wastes budget. The authors state plainly that this is consistent with existing evidence but has not been tested in a controlled comparison. Appendices add a computable LLM-based proxy SS-theta, a Fano bound capping branching factor at 2^((B+1)/(1-eps)) given B bits of routing information, an optimal branching factor of e under uniform cost, a depth bound L >= ceil(log_r(N/C)) tying corpus size N and context window C to required levels, and a coherence condition on pi (within-group affinity must exceed between-group affinity) that justifies top-down pruning. The whole framework assumes a static hierarchy; insertion, restructuring, and retrieval-driven decay break the Markov argument, which the authors name as the most pressing open problem.

For a personal knowledge backend built over conversational history, the paper supplies a design vocabulary rather than results: it says to choose summary style and search strategy jointly, that finer-grained extraction raises the retrieval ceiling, and that the depth bound predicts when one summary layer stops sufficing as a corpus grows toward organizational scale. Table 1's mapping of eleven systems onto (alpha, C, tau) is also a compact survey of the persistent-memory field as of early 2026.

Read from: full text

Wikidata: A Free Collaborative Knowledgebase

Vrandecic and Krotzsch, Communications of the ACM 57(10), 2014, DOI 10.1145/2629489

This is a systems and design article by Wikidata's project director and its data model lead, describing the collaboratively edited knowledgebase Wikimedia launched in October 2012 to hold Wikipedia's factual data in one place. The motivating problem is duplication: the same fact (the population of Rome, say) lived in hundreds of independently edited language articles and disagreed across them, while the underlying data, spread over 30 million articles in 287 languages, was nearly impossible to extract. The article walks through the design decisions (open editing, community-controlled schema, plurality of conflicting claims, secondary rather than primary data, multilingual by construction), the data model, and early growth statistics, and situates the project against Semantic MediaWiki, Freebase, OpenCyc, DBpedia, and YAGO.

The core representational contribution is a layered statement model rather than plain triples. Items carry property-value pairs, with a small set of datatypes (quantity, item, string, time, coordinates, URL); properties are themselves first-class pages with labels and declared datatypes, and the community, not a fixed ontology, decides which properties exist. Statements take subordinate property-value pairs called qualifiers, so "population of Rome = 2,651,040" can carry "as of 2010" and "method: estimation" attached to the assertion rather than the item. Every claim can carry references, themselves lists of property-value pairs, and contributors can mark statements preferred or deprecated to manage contradictory sources; the model also distinguishes "value unknown" and "no value" from mere incompleteness. By August 2014 this held 15.8 million items, 43.2 million statements (23.2 million with references), and 1,176 properties, with property use heavily skewed toward "instance of." The numbers document adoption, not data quality; the authors note the data is far from complete and complex query support did not yet exist.

For a personal knowledge backend built over conversational history, the transferable pieces are the qualifier-and-reference structure (every stored fact carries its temporal context and its source, which is exactly the provenance a memory system needs when conversations contradict each other), the preferred/deprecated mechanism for coexisting conflicting claims instead of forced resolution, and stable never-reused IDs decoupled from labels. The bootstrapping story also matters: Wikidata gained its initial item set by importing an existing artifact (language links), and about 90 percent of its half million daily edits came from bots, with human curation layered on top, a plausible division of labor for automated extraction plus owner review. The scope limit is that Wikidata's schema governance depends on a large volunteer community; a single-user or single-organization system inherits the representation but must supply curation by other means.

Read from: full text

Searching for Best Practices in Retrieval-Augmented Generation

Wang et al., EMNLP 2024, arXiv:2407.01219

The paper treats a RAG pipeline as a sequence of swappable modules (query classification, chunking, embedding, vector database, retrieval, reranking, repacking, summarization, generator fine-tuning) and searches for the best method at each position. Since testing every combination is infeasible, the authors run a greedy three-step procedure: compare candidate methods per module in isolation, then optimize one module at a time within a full pipeline while holding the others fixed, then assemble recommended configurations. The end-to-end evaluation uses a Llama2-7B-Chat generator over a Milvus index of 10 million English Wikipedia passages plus 4 million medical texts, scored across commonsense reasoning, fact checking, open-domain QA, multihop QA, and medical QA, with RAGAs-style RAG metrics, sub-sampled to at most 500 examples per dataset.

The per-module findings come with specific conditions. Sentence-level chunking at 512 tokens scored best for faithfulness (97.59) on a corpus of just sixty pages of a Lyft 10-K, so the chunk-size result is thin evidence. LLM-Embedder matched BAAI/bge-large-en on MS MARCO retrieval at a third the size. On TREC DL19/20, HyDE plus hybrid search (BM25 weighted at alpha 0.3 against dense scores) gave the best retrieval quality, while query rewriting and decomposition did not help. For reranking on MS MARCO, monoT5 balanced quality and cost; TILDEv2 was 225 times faster (0.02 versus 4.5 seconds per query) at some accuracy loss. In the full pipeline, query classification (a BERT classifier deciding whether to retrieve at all, 0.95 accuracy) raised the average score from 0.428 to 0.443 and cut latency from 16.41 to 11.58 seconds per query. Reverse repacking, putting the most relevant document nearest the query, beat forward and sides. Fine-tuning the generator on a mix of one relevant and one random document made it most resistant to retrieval noise. The two recipes: best performance uses Hybrid with HyDE, monoT5, reverse, Recomp (average 0.483, about 11.7 s/query); the balanced recipe swaps in plain hybrid retrieval and TILDEv2, cutting latency to roughly 1.5 seconds at similar quality.

For a conversational-history backend, the transferable lessons are the deciding-when-to-retrieve gate (the single cheapest win in both accuracy and latency), hybrid sparse-plus-dense retrieval with a light reranker, and ordering context so the best evidence sits nearest the query. The caveat matters for memory work generally: these numbers come from public QA corpora and a 7B generator, and the paper itself concedes chunking was only lightly explored, so the recipes are starting priors rather than settled law for personal or organizational memory stores.

Read from: pages 1-16 of 22

Same Author or Just Same Topic? Towards Content-Independent Style Representations

Wegmann, Schraagen, Nguyen 2022, Same Author or Just Same Topic?, RepL4NLP 2022, arXiv:2204.04907

The paper attacks a confound in learning writing-style embeddings: models trained on authorship verification (AV, deciding whether two texts share an author) can score well by memorizing what an author writes about rather than how. The authors propose two changes to the training task. First, a contrastive AV setup (CAV): instead of a binary pair, the model sees an anchor utterance and must pick which of two candidates shares its author, trained with a triplet loss. Second, content control (CC): the different-author distractor is sampled from the same conversation as the anchor, from the same domain (subreddit), or at random, so that at the strictest level topic can no longer separate authors. They fine-tune siamese BERT, cased BERT, and RoBERTa models (RoBERTa consistently won on dev) on 210k training tasks built from a 2018 Reddit sample of 100 English-language subreddits, 600 conversations each, with a 70/15/15 author-disjoint split.

Conversation-controlled tasks are the hardest form of AV: every model scores lowest on that test set (around .69 AUC for models trained on it) and highest on the random one (up to .79 for the no-control model), consistent with the distractors carrying fewer content cues. The decisive evidence comes from a new STEL-Or-Content task, which pits a same-style paraphrase against a same-content, different-style sentence. Base RoBERTa picks style only 5 percent of the time; the CAV model with conversation control reaches .42 accuracy on the formal/informal dimension, against .24 or less for domain or no control. The authors argue this is not mere punishment of lexical overlap, since the same model holds even AV performance across all three test sets and normal STEL performance. Agglomerative clustering of its embeddings (7 clusters, 14,756 utterances) surfaces concrete stylistic signatures: terminal-punctuation habits, lowercase i, contraction spelling, apostrophe versus right single quotation mark, line breaks. All of this is one model family on one platform in one language; the conversation label also does not transfer directly to non-conversational corpora, a limit the paper states itself.

For a personal knowledge backend built over conversational history, the operative lesson is that “same source” signals are contaminated: any retrieval or user-modeling layer that embeds utterances will silently entangle who is speaking with what they discuss, and disentangling requires deliberately constructed hard negatives, here obtained for free from conversation structure. That trick, mining supervision from the thread a message sits in, scales to organizational chat archives without labeling. The paper also models an evaluation discipline that persistent-memory work mostly lacks: do not trust the training metric; build a probe where the shortcut and the target answer disagree.

Read from: full text

Machine Knowledge: Creation and Curation of Comprehensive Knowledge Bases

Weikum, Dong, Razniewski, Suchanek; Foundations and Trends in Databases, 2021; arXiv:2009.11564

This is a 289-page survey of how large knowledge bases (KBs) are built and maintained automatically from web and text sources. The authors organize the field into a pipeline of four phases: discovery (source selection, entity detection, taxonomy construction), canonicalization (entity linking and matching), augmentation (extracting attributes and relations, then growing an open schema when the fixed one runs out), and cleaning (constraint reasoning, rule mining, embeddings, provenance and temporal scoping). Each phase gets a methods chapter, ranging from hand-written regex patterns and distant supervision through CRFs, graph algorithms, and transformer-based extractors, followed by case studies of YAGO, DBpedia, NELL, Wikidata, and industrial graphs at Google and Amazon.

What the survey establishes is less a single result than a set of engineering conclusions drawn across two decades of systems. The quality bar they set is error rates below 5 percent, ideally under 1 percent, and they show the two goals of correctness and coverage pull in opposite directions: conservative extraction from premium sources (Wikipedia, IMDB, MayoClinic) gets the low-hanging fruit, then that initial KB seeds distant supervision for riskier open extraction. Public KBs hold millions of entities and up to billions of statements; proprietary ones run one to two orders of magnitude larger. Entities and types must come first, because attributes and relations are only as good as the canonicalized backbone under them. And no KB construction is a one-shot task: it is a life-cycle with humans (architects and curators) permanently in the loop, since no single method wins across the precision, recall, and cost trade-offs.

Two threads bear directly on a personal memory backend over conversational history. First, the wrap-up names the personal KB as an open research problem: an “augmented memory” on the user’s own device, built from a longitudinal digital footprint, with unresolved questions about what to capture, how to contextualize statements for interpretability, and how users manage the life-cycle. That is nearly a spec for a conversation-derived memory tree, and it inherits the survey’s machinery (canonicalization so one entity has one node, temporal scoping so facts age, completeness assessment so the system knows what it does not know). Second, the language-model discussion is a useful caution: they show a 2021-era model completes “Paris is located on the banks of the [?]” correctly but fails on Dylan’s protest songs, results they call “sort of correct, but completely useless,” which is their argument that latent model knowledge cannot replace an explicit, curated store. Scoped to pre-GPT-3-era models, but the structural point about verifiable symbolic memory versus parametric recall still frames the field.

Read from: pages 1-20 and 229-236 of 289

Recursively Summarizing Books with Human Feedback

Wu et al. (OpenAI Alignment team), arXiv preprint, 2021, arXiv:2109.10862

The paper trains GPT-3 models (6B and 175B parameters, 2048-token context) to summarize entire fiction novels by fixed recursive task decomposition: a chunking algorithm splits the book, the model summarizes each chunk, then summarizes concatenations of those summaries, recursing to depth 3 for books of hundreds of thousands of words. Each task is conditioned on prior summaries at the same level ("previous context") so summaries flow in order. A single model handles every level, trained first by behavioral cloning on human demonstrations, then by reinforcement learning against a reward model fit to human comparisons, following Stiennon et al. (2020). The framing is scalable oversight: labelers judge each subtask against only its local input, so no one has to read the whole book, which the authors estimate makes each data point over 50x cheaper than end-to-end collection.

On 40 popular books published in 2020 (unseen in pretraining, primarily narrative fiction averaging over 100K words), the best 175B RL model produced summaries rated 6 of 7 about 5 percent of the time and 5 of 7 over 15 percent, but the average stayed well below the human-written summaries; labeler agreement on relative model quality was near 80 percent. RL clearly beat behavioral cloning at 175B, less so at 6B, and comparisons became more label-efficient than demonstrations past 10-20K labels while being 3x faster to collect. Training only on the first subtree generalized as well as training on the full tree; the final full-tree model actually got worse, which the authors tie to compounding lower-level errors and auto-induced distributional shift. The 175B models set state of the art on BookSum (beating non-oracle baselines by 3-4 ROUGE points and every baseline on BERTScore), and their depth 1 summaries let a zero-shot 3B UnifiedQA model score competitively on NarrativeQA.

For a personal knowledge backend built over conversational history, this is the canonical evidence that hierarchical summarization can compress arbitrarily long corpora while keeping facts traceable back to source text, and that summary trees support downstream question answering without retrieval over raw text. Its failure modes are equally instructive: local context produced a Pride and Prejudice summary that turned a request for a dance into a marriage proposal, book-level output read as event lists rather than coherent narrative, and themes spread thinly across many chunks vanished. Any memory tree that distills leaves independently inherits exactly these risks (within this study's scope: fiction, GPT-3-era models, fixed decomposition), so cross-leaf context and error auditing at composition points are design requirements, not refinements.

Read from: pages 1-20 of 37

MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions

Yuanzhe Hu, Yu Wang, Julian McAuley (UC San Diego), ICLR 2026, arXiv:2507.05257

The paper builds a benchmark for what it calls memory agents: LLM systems that accumulate information across many turns rather than reading one long document. Drawing on memory science, the authors name four competencies a memory system needs: accurate retrieval, test-time learning, long-range understanding, and selective forgetting. Their core methodological move is to feed context incrementally, chunk by chunk, wrapped in user messages with explicit memorize-this instructions, so that the agent's storage mechanism actually fires; questions come only after ingestion is complete. They restructure existing long-context datasets (document QA, LongMemEval, intent classification, novel summarization) and add two of their own, EventQA for temporal recall over novels and FactConsolidation for overwriting facts with later contradictory ones. The result is 2,071 questions over contexts of 103K to 1.44M tokens, run against long-context agents, three flavors of RAG (BM25, embedding models, structure-augmented systems like GraphRAG and HippoRAG-v2), and agentic memory products including MemGPT, MIRIX, Mem0, Cognee, and Zep, most with GPT-4o-mini as backbone.

No method masters all four competencies. RAG agents beat their raw backbone on accurate retrieval (HippoRAG-v2 averages 65.1 against GPT-4o-mini's 49.2), but long-context models win test-time learning and long-range understanding, where retrieving top-k passages cannot deliver a global view; GraphRAG scores 0.4 on novel summarization. Selective forgetting breaks everyone: no system exceeds 28 percent on multi-hop fact updates, and even o4-mini, which reaches 80 on a 6K-token version, collapses to 14 at 32K, so the failure is length-driven, not task-driven. The commercial memory products fare worst overall (Mem0 21.1, Cognee 20.6, Zep 24.0, against 60.6 for plain long-context GPT-5-mini). Ablations show smaller chunks help retrieval but hurt holistic tasks, and a stronger backbone lifts MIRIX by 9.7 points while barely moving simple RAG. Scope caveats: one backbone family dominates the main table, and budget limited which agents were tested.

For anyone building a personal knowledge backend over conversational history, the incremental-ingestion protocol is exactly the deployment shape, and the findings are cautionary in a specific way: retrieval-first architectures, which is what a summarized archive with search amounts to, handle lookup well but fail at superseding stale facts and at questions requiring the whole corpus. That argues for explicit update-and-consolidation machinery (newest-wins provenance, periodic re-summarization) rather than append-and-retrieve alone. For the field, the paper supplies a shared vocabulary of four competencies and hard evidence that current commercial memory layers lag a plain long context, which reframes where the open problems actually are.

Read from: pages 1-10 of 33

Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering

Xu, Cruz, Guevara, Wang, Deshpande, Wang, and Li (LinkedIn), SIGIR 2024 industry track, arXiv:2404.17723

This is a five-page industry paper describing a question-answering system deployed inside LinkedIn's customer service organization. The problem is retrieval over historical issue-tracking tickets (Jira), where conventional RAG treats tickets as plain text, chunks them to fit embedding-model context windows, and thereby loses two things: the internal structure of each ticket and the explicit links between tickets. The authors replace the flat text corpus with a knowledge graph built in two phases. Intra-ticket parsing turns each ticket into a tree whose nodes are sections (summary, description, steps to reproduce, priority, root cause, comments), using rule-based extraction for predefined fields and an LLM guided by a YAML template for the rest. Inter-ticket connection then links trees by explicit references recorded in Jira (clone, related-to) and by implicit edges added when the cosine similarity of ticket-title embeddings crosses a threshold. Node text is embedded with pretrained models (E5, BERT) into a vector database (Qdrant); the graph lives in Neo4j.

At query time an LLM parses the user's question into named entities keyed to template sections plus an intent (for example, steps to reproduce). Entity values are matched against section-specific node embeddings, node scores are summed to the ticket level to rank the top K tickets, and the LLM then rewrites the query with the winning ticket ID and translates it into a Cypher query that walks the tree to the intent node. A fallback reverts to plain text retrieval if query execution fails. On a curated golden dataset, with GPT-4 and E5 held fixed in both arms, the method lifts MRR from 0.522 to 0.927 (a 77.6 percent gain), Recall@3 from 0.640 to 1.000, and BLEU from 0.057 to 0.377. In a randomized split of the support team over roughly six months, median per-issue resolution time fell 28.6 percent (P50 from 7 to 5 hours, mean from 40 to 15). The golden dataset is small and internal, the graph template is hand-built, and the benchmark numbers reflect one domain, so the figures establish a deployment result rather than a general benchmark.

For a personal knowledge backend over conversational history, the transferable idea is the dual-level representation: parse each unit (here a ticket, there a session or leaf) into a section tree, then connect units by explicit references and thresholded embedding similarity, so retrieval can return a coherent subgraph instead of arbitrary chunks. The section-aware scoring, which sums similarity only over nodes of the queried type, is a concrete mechanism for intent-scoped retrieval, and the paper's own future-work list (automatic template extraction, dynamic graph updates from queries) marks exactly the parts that were manual here and would need to be automatic at organizational scale.

Read from: full text

A Comprehensive Survey on Automatic Knowledge Graph Construction

Zhong, Wu, Li, Peng, and Wu; ACM Computing Surveys 56(4), 2024; arXiv:2302.05019

This is a survey of more than 300 methods for building knowledge graphs automatically from raw data. The authors organize construction into three stages: knowledge acquisition (entity discovery, coreference resolution, relation extraction), knowledge refinement (graph completion and fusion with other graphs), and knowledge evolution (conditional and temporal knowledge, dynamics over time). They define a knowledge graph as G = {E, R, T, F_k}, where T is a set of triples (head, tail, relation) and F_k is background knowledge, a rule set or schema that constrains which facts count as knowledge rather than mere data; a graph without this component, they argue, is just a data graph. The survey frames its coverage with the HACE view of big data (heterogeneous, autonomous, complex, evolving) and compares itself against nine prior surveys, claiming broader coverage of conditions, coreference, and semi-structured sources.

The construction-from-text sections lay out a stable pipeline vocabulary. Entity discovery splits into named entity recognition, fine-grained entity typing, and entity linking (disambiguation against existing nodes, creating a node when none exists). Coreference resolution runs as mention detection followed by antecedent scoring, with the end-to-end span-ranking model of Lee et al. as the standard deep architecture. Relation extraction divides by configuration: open extraction without a schema, schema-bound classification, distant supervision with its noise problem and the at-least-one multi-instance assumption, plus document-level, few-shot, and joint variants. For each task the survey tracks the same progression, from rules and wrappers through statistical models (CRFs, SVMs) to CNN, LSTM, attention, GCN, and pre-trained-language-model designs. It also catalogs practical resources: about two dozen KG projects with sizes (Wikidata at 96M+ items, YAGO at 2B+ facts, ConceptNet at 34M+ items) and tools from spaCy and OpenNRE to the end-to-end gBuilder.

For a personal knowledge backend built over conversational history, the useful contributions are the pipeline decomposition itself (a memory system doing entity linking and coreference over chat logs is running exactly these tasks), the insistence that a schema or background knowledge separates a knowledge graph from a data graph, and the storage discussion (relational, key/value, and native graph stores such as Neo4j and RDF systems). The open problems the authors flag map directly onto that setting: long intricate contexts requiring coreference, logic, and commonsense reasoning; knowledge dynamics as facts change; and federated learning for privacy when scaling to an organization, where encrypted entity alignment remains unsolved. The survey predates LLM-based extraction, so its method coverage stops at BERT-era models.

Read from: pages 1-20 (introduction, definitions, resources, preprocessing, entity discovery, coreference, relation extraction through distant supervision) and pages 33-37 (knowledge dynamics, storage, discussion, conclusion); skipped pages 21-32 (few-shot, document-level, and joint extraction; knowledge refinement; most of knowledge evolution) and the references.